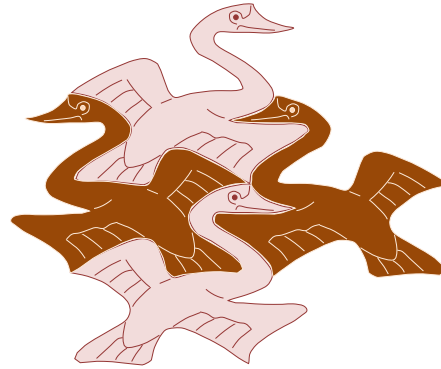




Design Patterns

Software Design & Architecture

Version 3.9.0
© Marco Torchiano, 2026



Licensing Note



This work is licensed under the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

To view a copy of this license, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

You are free: to copy, distribute, display, and perform the work

Under the following conditions:



Attribution. You must attribute the work in the manner specified by the author or licensor.



Non-commercial. You may not use this work for commercial purposes.

No Derivative Works. You may not alter, transform, or build upon this work.



For any reuse or distribution, you must make clear to others the license terms of this work.

- Any of these conditions can be waived if you get permission from the copyright holder.

Your fair use and other rights are in no way affected by the above.

Pattern

A reusable **solution**
to a known **problem**
in a well defined **context**

...just one of the possible definitions

Pattern

- Context
 - A (design) situation giving rise to a (design) problem
- Problem
 - Set of forces repeatedly arising in the context
 - Force: any relevant aspect of the problem (E.g., requirements, constraints, desirable properties)
- Solution
 - A proven resolution of the problem
 - Configuration to balance forces
 - Structure with components and relationships
 - Run-time behaviour

Example

*SOCIAL
PATTERN*

- Context:
 - At the supermarket several customers crowd the gastronomy desk to get their fresh cut of ham
- Problem:
 - Customers quarrel to have their turn first
 - Order of arrival should be obeyed
 - It is hard to spot who arrived earlier or later
- Solution:
 - Provide numbered tickets the customer take as soon as they arrive and which they are called by

History

- Initially proposed by Christopher Alexander
- He described patterns for architecture (of buildings)
 - *The pattern is, in short, at the same time a thing, which happens in the world, and the rule which tells us how to create that thing and when we create it. It is both a process and a thing ...*

Types of Software Patterns

- Architectural Patterns
 - Address system wide structures
- Design Patterns
 - Leverage higher level mechanisms
- Idioms
 - Leverage language specific features

Architectural pattern

- Expresses a fundamental structural organization schema for software systems
- Provides a set of predefined components with their responsibilities
- Defines the rules and guidelines for organizing the relationships between the components

Example

ARCHITECTURAL PATTERN

- Context:
 - several programs that are used in sequence read from input and write sequentially to output
- Problem:
 - there are a lot of intermediate files used for communication between programs
- Solution:
 - adopt a pipe & filter architecture feeding a program with the result of the previous one

Design pattern

- Provides a scheme for refining components of a software system or their relationships
- Describes a commonly recurring structure of communicating components

Example

DESIGN
PATTERN

- Context:
 - A class library providing few functionalities contains a lot of classes
- Problem:
 - The user is exposed to the internal complexity of the library
- Solution:
 - Create a new façade class that interacts with the user and hide all the details

Idiom

- It is a low-level pattern specific to a programming language
- Describes how to implement particular aspects of components or the relationships between them
- Leverages the features of a programming language

Example

IDIOM

- Context:
 - An attribute is constant and should be globally available to many classes
- Problem:
 - Opening access would allow unauthorized modifications
 - The attribute is repeated in every object
- Solution:
 - Make it public static final

Pattern Description

Canonical (Coplien) Form	GoF Form
Name	Name
Alias	Also Known as
Problem	Intent
Context	Applicability
Forces	Motivation
Solution	Participants, Structure Collaborations Implementation
Example	Sample Code
Resulting Context	Consequences
Rationale	
Known Uses	Known Uses
Related Patterns	Related Patterns

Pattern language

- Pattern do not exist in isolation
 - Two or more patterns are applied together
 - A pattern is used to implement part of another pattern
 - A pattern can introduce a problem solved by another
- They are called Pattern Languages
 - Or pattern systems

Pattern Language

- Collection of patterns together with guidelines for
 - Implementation
 - Combination
 - Practical use
- Should
 - Count enough patterns
 - Describe patterns uniformly
 - Present relationships

Idioms

X-ability

- Context
 - Dealing with many heterogeneous objects
- Problem
 - Perform a common operation on them, e.g. printing
- Solution
 - Derive from a class or interface that defines the required methods

Text representation

- In Java:
 - All methods extends the root class **Object** the provides the method **toString()**
 - Default: `it.polito.Person@2f4ee98`
- In Python
 - Classes automatically have methods **__str__()** and **__repr__()**
 - Default: `<__main__.Person object at 0x7f8c1c2d>`

Comparability

- In Java:
 - Classes to be compared implement Comparable and provide method `compareTo()`
- In Python:
 - Classes need to implement the methods:
 - `__lt__(self, other)` # <
 - `__le__(self, other)` # ≤
 - `__gt__(self, other)` # >
 - `__ge__(self, other)` # ≥

Strong typing vs. Duck Typing

If it looks like a duck, swims like a duck, and quacks like a duck, then it probably is a duck.

- Strong typed language
 - You must define interfaces and classes to be able to call a specific method
 - Compile time type checking
- Dynamic / duck typed language
 - You just add a method to a class and you can call it
 - No compile time type checking

Class Flags

- Context
 - There are operations conditional to some condition (or flag) for the objects
- Problem
 - We need to enable the algorithm to detect the presence of flag on the objects treated without forcing them to provide a method that can be checked

Class Flags – solution

- Implement an interface (in Java)
 - Interface `Serializable` enable writing the objects to an `ObjectStream`
 - Interface `Cloneable` flags a class a safe for making a field-by-field copy of instances
- Add annotation to the class
 - JPA uses the `@Entity` annotation to mark a class as persistent

Property

- A property is a **named** characteristic of an object
- It is accessed through well-defined operations called **accessors**.
 - typically getter and optionally setter methods
 - forming part of the object's public interface
 - it is not necessarily an attribute

Property accessor idioms

- getter/setter using "bean" convention
- annotations
- attribute specific **role** interface
- general purpose **map-like** methods
- config objects

Java Bean Convention

- For a property `Prop` define:
 - Getter: `getProp()`
 - Setter: `setProp(T value)`
 - optional

Java Bean Convention

```
public class Person {  
    private String name;  
    private String email;  
    private LocalDate birth;  
    public String getName(){ return name; }  
    public String getEmail(){ return email; }  
    public void setEmail(String e) { email = e; }  
    public int getAge(){  
        return Period.between(birth,LocalDate.now())  
            .getYears();  
    }  
}
```

Java Bean Access

Using Java reflection API:

```
static Object getProperty(Object o, String name)
    throws Exception {
    return o.getClass()
        .getMethod(getter(name), null)
        .invoke(o, null);
}
```

Java Annotation

```
public class Person {  
    @Property  
    private String name;  
    @Property(name="e-mail")  
    private String email;  
    private LocalDate birth;  
    @Property  
    int getAge(){ ... }  
}
```

Java Annotation

```
...  
Field f = field.get();  
try{  
    f.setAccessible(true);  
    Object value = f.get(o);  
    return value;  
}finally{  
    f.setAccessible(false);  
}
```

Python annotation

```
class Person:
    @property
    def name(self) → str:
        return self._name
    @property
    def email(self) → str:
        return self._email
    @email.setter
    def email(self, value: str) → None:
        self._email = value
    @property
    def age(self) → int:
        years = date.today().year - self._birth.year
        return years
```

Special role-interface

```
public interface HasAge {  
    int getAge();  
}
```

```
public class Person implements HasAge {  
    ...  
    @Override  
    public int getAge(){ ... }  
}
```


General map-like methods

```
public class Person {  
    public Object getProperty(String name){  
        switch (name) {  
            case "name": return name;  
            case "e-mail": return email;  
            case "age": return getAge();  
            default: throw new NoSuchElementException(name);  
        }  
    }  
    public void setProperty(String name, Object value){  
        if(name.equals("e-mail") email = (String)value;  
        else throw new NoSuchElementException(name);  
    }  
}
```

Fluent Interfaces

- Method to design OO API based on extensive use of **method chaining**
- The goal is to improve readability
 - Code looks like prose
 - Often used to build complex objects
- Create a sort of Domain Specific Language (DSL) leveraging the syntax of the host language

See: <https://www.martinfowler.com/bliki/FluentInterface.html>

Fluent Interface

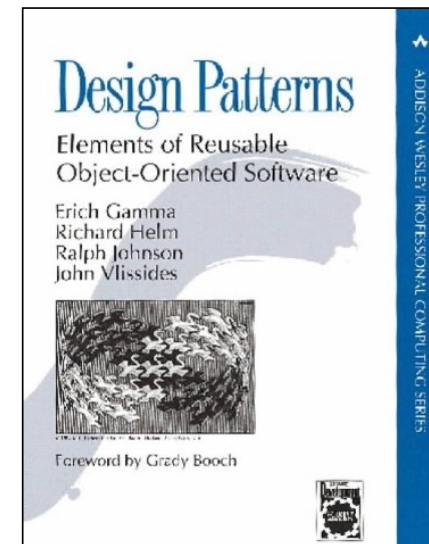
- In Java, `Stream` and `Optional` provide fluent APIs

```
String result =  
    Optional.of("hello")  
        .filter(s → s.length() > 3)  
        .map(String::toUpperCase)  
        .orElse("default");
```

Design Patterns

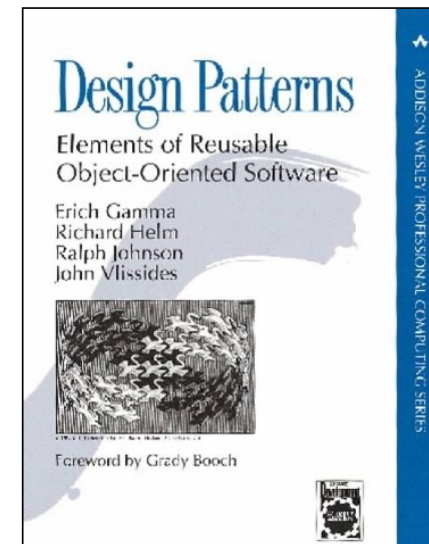
Design Patterns (GoF)

- Describe the structure of components
- Most widespread category of pattern
- First category of patterns proposed for software development



Design Patterns (GoF)

- Creational
 - E.g. Abstract Factory, Singleton
- Structural
 - E.g. Façade, Composite
- Behavioral
 - Class: e.g. Template Method
 - Object: e.g. Observer



Design patterns

- Description of communicating objects and classes that are customized to solve a general design problem in a particular context
- A design pattern names, abstracts, and identifies the key aspects of a common design structure that make it useful for creating a reusable object-oriented design

Pattern classification

- Purpose
 - Creational
 - Structural
 - Behavioral
- Scope
 - Class
 - Object

Pattern classification

		Purpose		
		Creational	Structural	Behavioral
Scope	Class	1	1	2
	Object	4	6	10

Pattern selection

- Consider how patterns solve problems
- Scan intent sections
- Study how pattern interrelate
- Study patterns of like purpose
- Examine a cause of redesign
- Consider what should be variable in your design

Using a pattern

- Read through the pattern
- Go back and study
 - Structure
 - Participants
 - Collaborations
- Look at the sample code

Using a pattern

- Choose names for participants
 - Meaningful in the application context
- Define the classes
- Choose operation names
 - Application specific
- Implement operations

Creational

Design Patterns

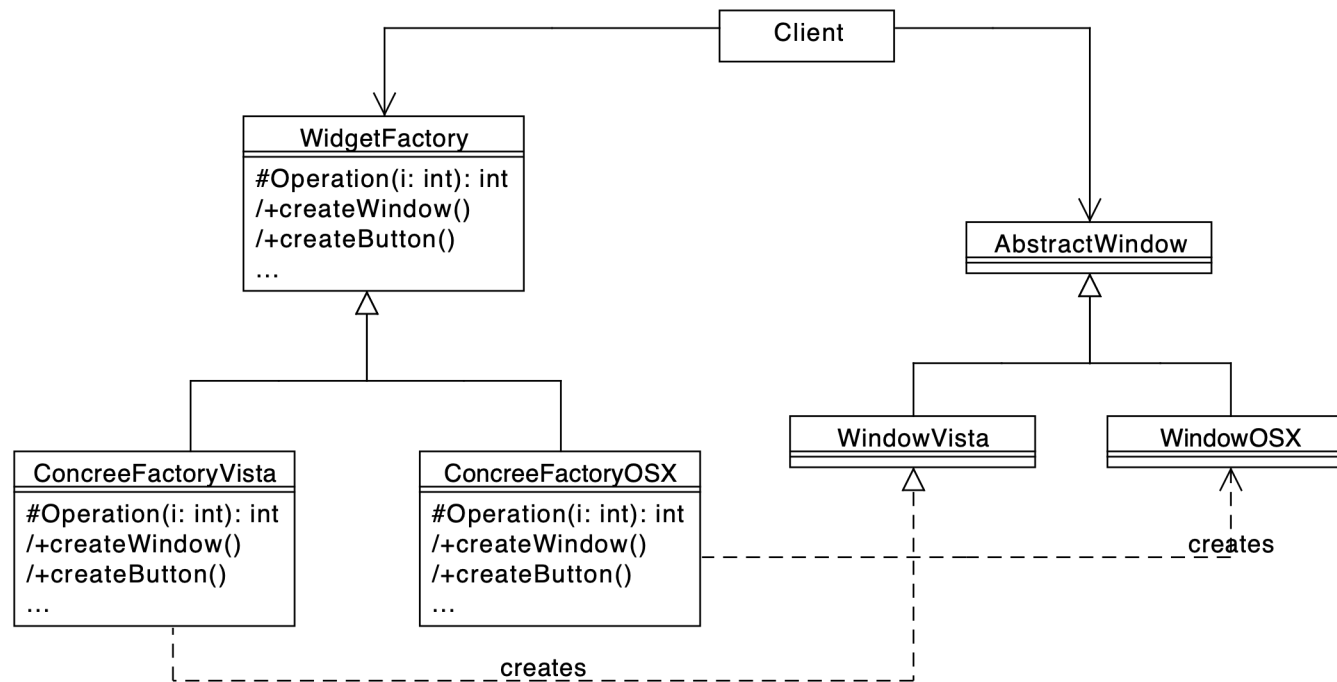
Creational patterns

- Class:
 - Factory Method
- Object
 - Abstract Factory
 - Builder
 - Prototype
 - Singleton

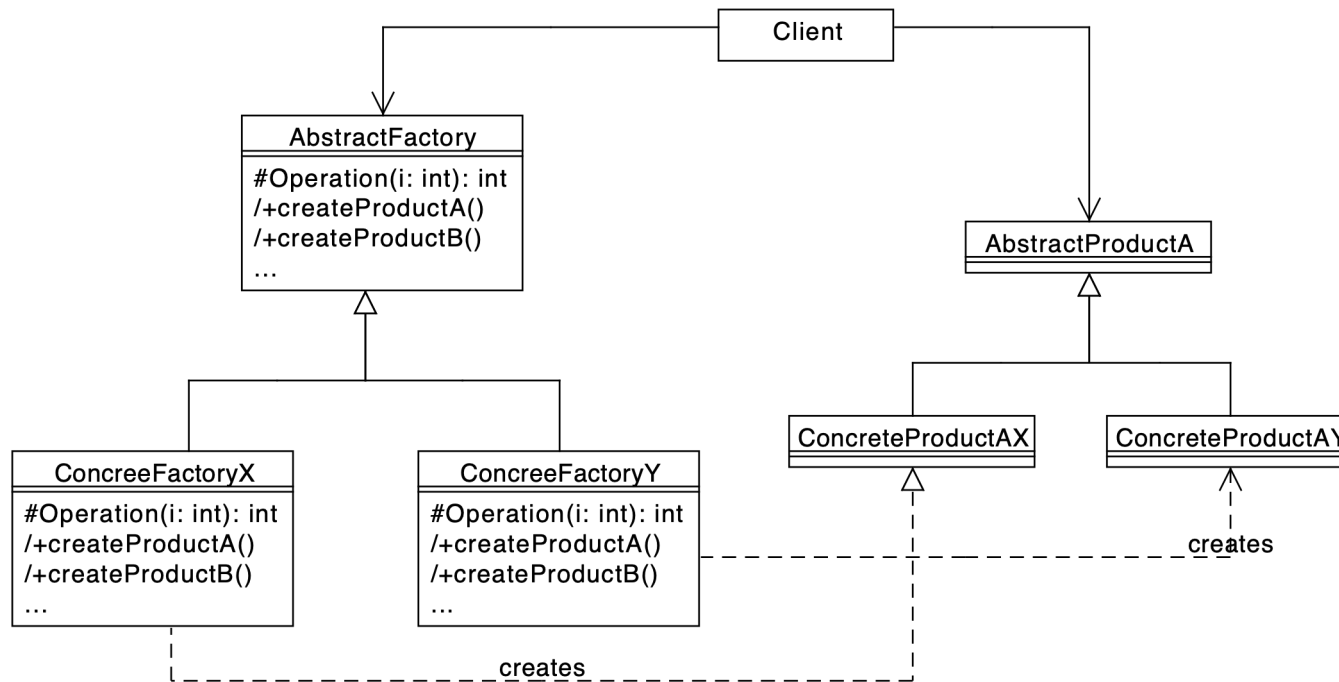
Abstract Factory

- Context
 - A family of related classes can have different implementation details
- Problem
 - The client should not know anything about which variant they are using / creating

Abstract Factory Example



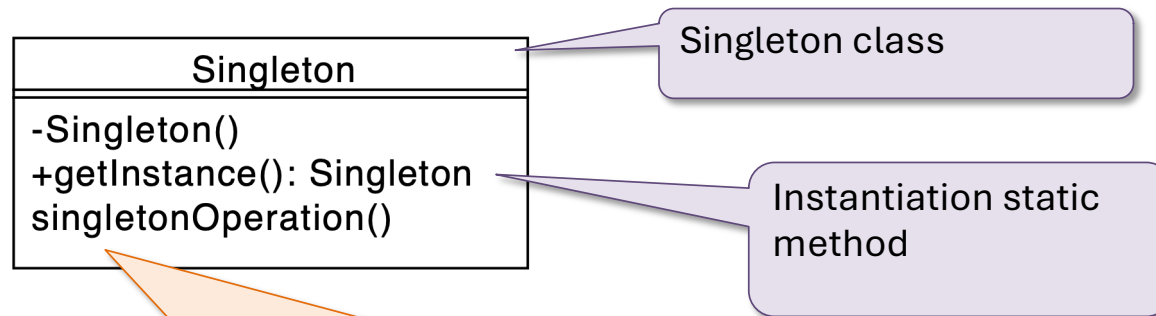
Abstract Factory



Singleton

- Context:
 - A class represents a concept that requires a single instance
- Problem:
 - Clients could use this class in an inappropriate way

Singleton Pattern



```
private Singleton() { }  
private static Singleton instance;  
public static Singleton getInstance(){  
    if(instance==null)  
        instance = new Singleton();  
    return instance;  
}
```

Singleton Example

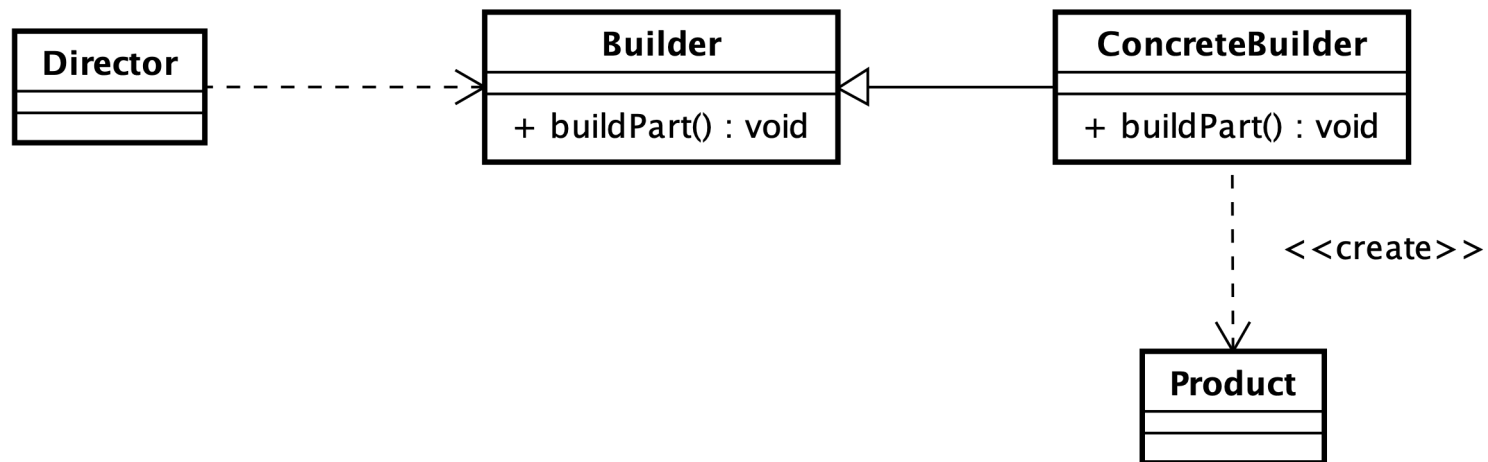
- Java `java.awt.Toolkit`
 - Singleton + FactoryMethod

java.awt::Toolkit
-Toolkit() +getDefaultToolkit(): Toolkit ...

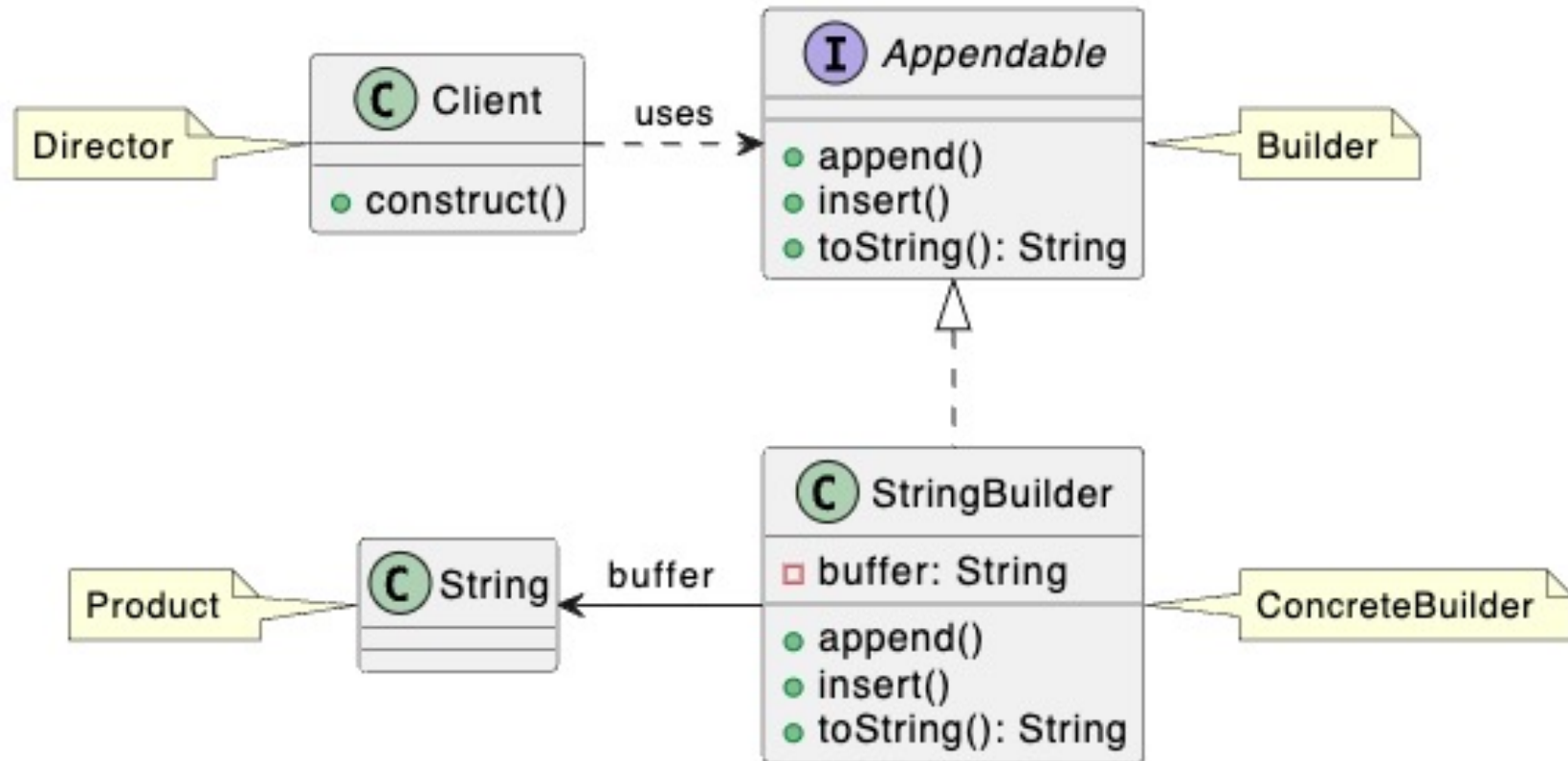
Builder

- Context
 - An object of a complex class has to be created
- Problem
 - The creation entails complex interaction with the object
 - Different variation of the target object might be created

Builder Pattern



Builder example



Example

- Usual non-fluent

$$10.40 \text{ kg} \cdot \text{m}^2 \cdot \text{s}^{-3}$$

```
Measure power = new Measure(10.4);  
power.addUnit("kg", 1);  
power.addUnit("m", 2);  
power.addUnit("s", -3);  
power.setPrecision(2);
```

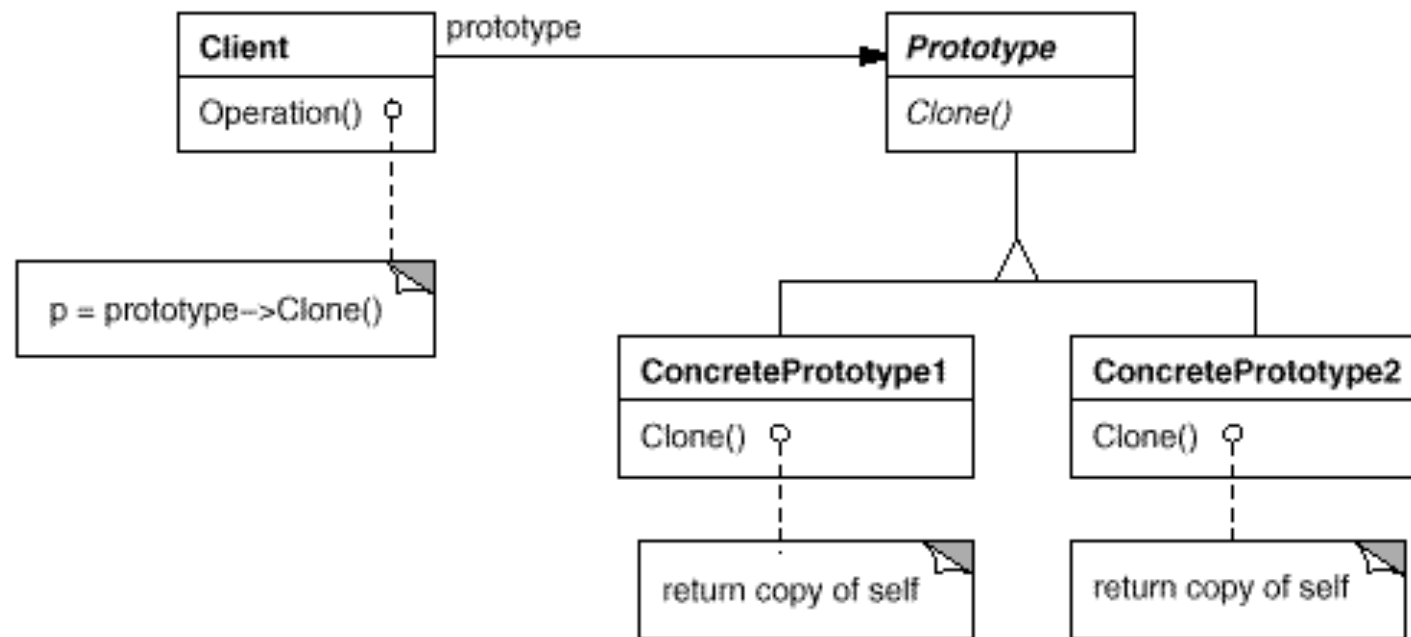
- Fluent builder

```
Measure power = Measure.value(10.4).  
    is("kg").by("m").squared().by("s").to(-3).  
    withPrecision(2).done();
```


Prototype

- Context
 - Several object can be created out of a palette of similar classes
- Problem
 - The class hierarchy can become quite complex
 - Classes in hierarchy are differentiated by minor parameters (that could be defined through constructor arguments)

Prototype



Structural

Design Patterns

Structural patterns

- Structural patterns are concerned with how classes and objects are composed to form larger structures.

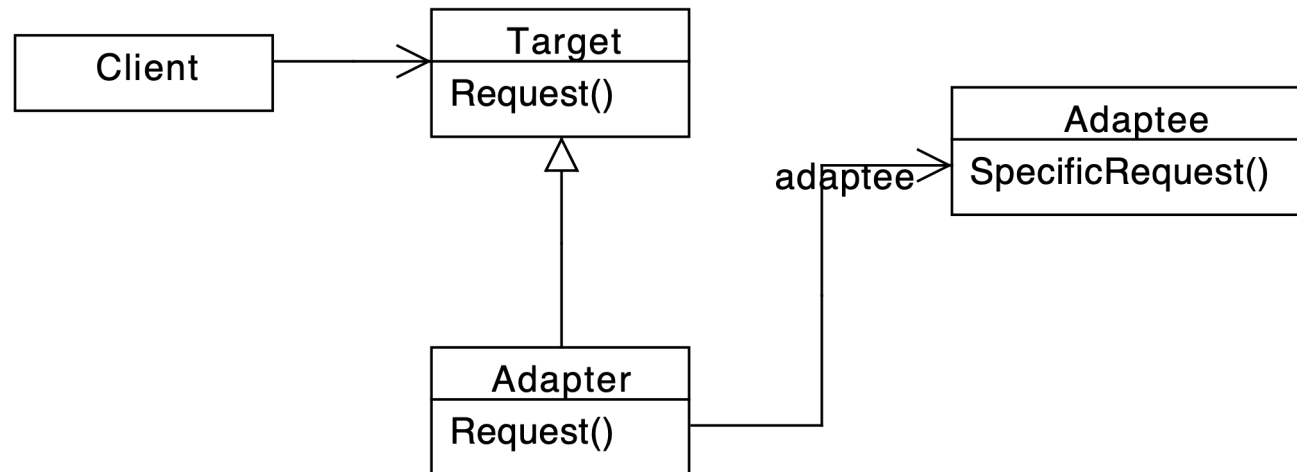
GoF structural patterns

- Adapter
- Bridge
- Composite
- Decorator
- Facade
- Flyweight
- Proxy

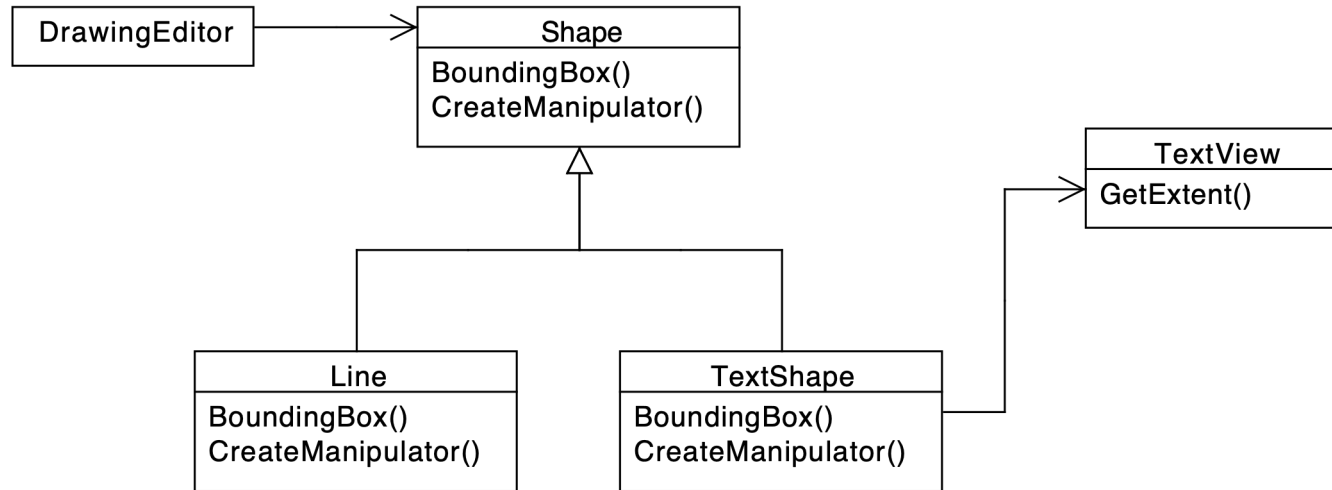
Adapter

- Context:
 - A class provides the features required by another class but its interface is not the one expected
- Problem:
 - The integration of the provider class should be possible without modifying it
 - Its source code could be not available
 - It is already used as it is somewhere else

Adapter



Adapter example



Java Listener Adapter

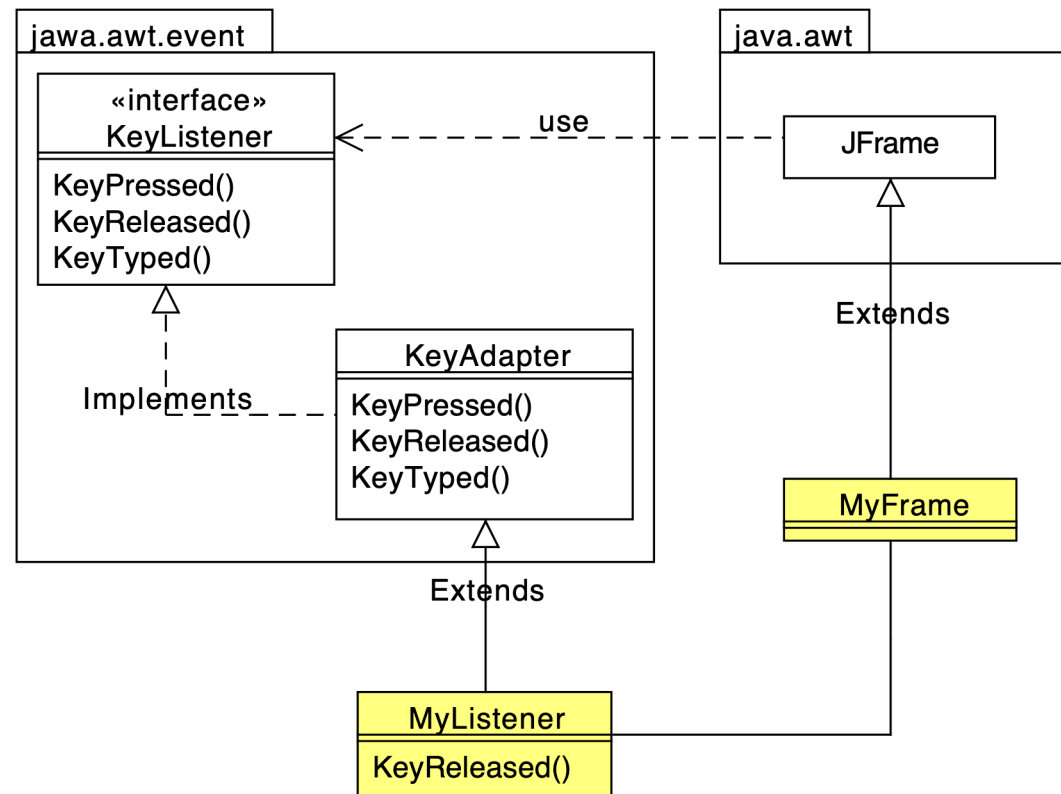
- In Java GUI, events are handled by Listeners
- Listener classes need to implement Listener interfaces
 - Include several methods
 - They all should be implemented

Java Listener Adapter

```
class MyListener{
    public void KeyPressed(..){}
    public void KeyReleased(..){
        // ... handle event
    }
    public void KeyTyped(..){} }
```

```
class MyListener{
    public void KeyReleased(..){
        // ... handle event
    }
}
```

Java Listener Adapter



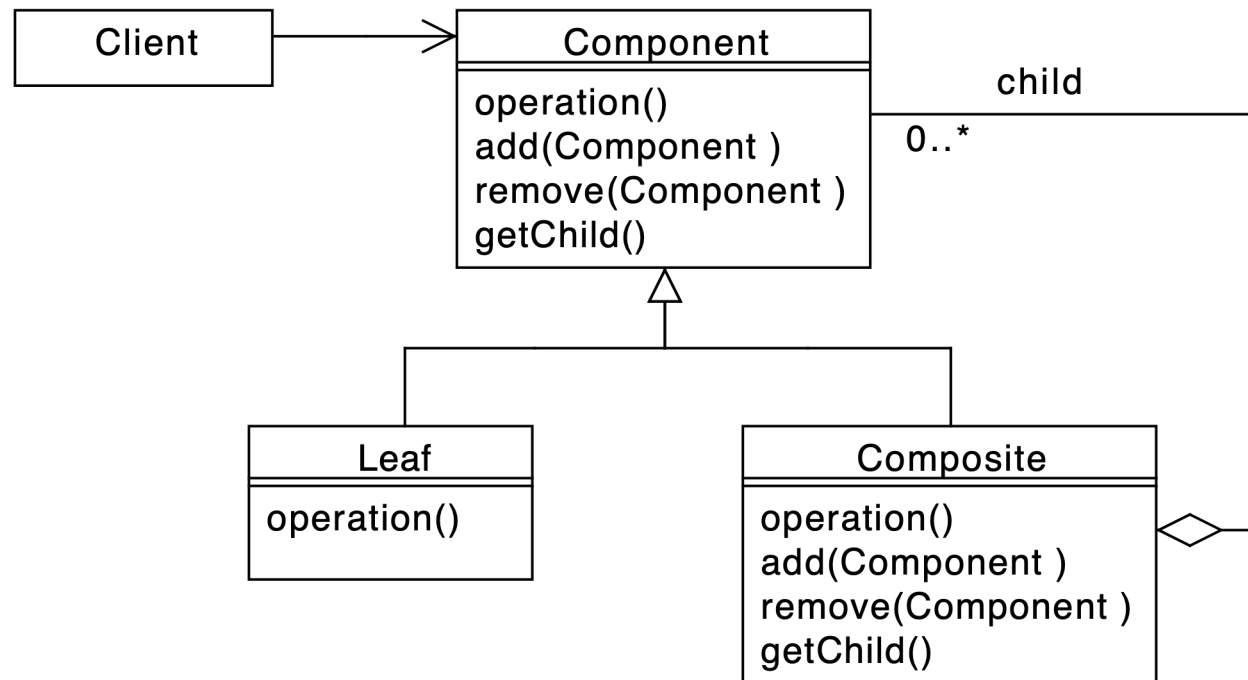
Adapter

- Adapter pattern
 - Inheritance plays a fundamental role
 - Only example of structural class pattern

Composite

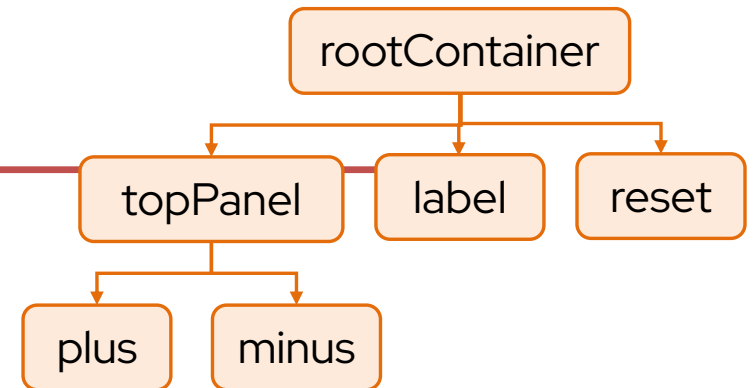
- Context:
 - You need to represent part-whole hierarchies of objects
- Problem
 - Clients are complex
 - Difference between composition objects and individual objects.

Composite



Composite example

- Java UI Component and Composite

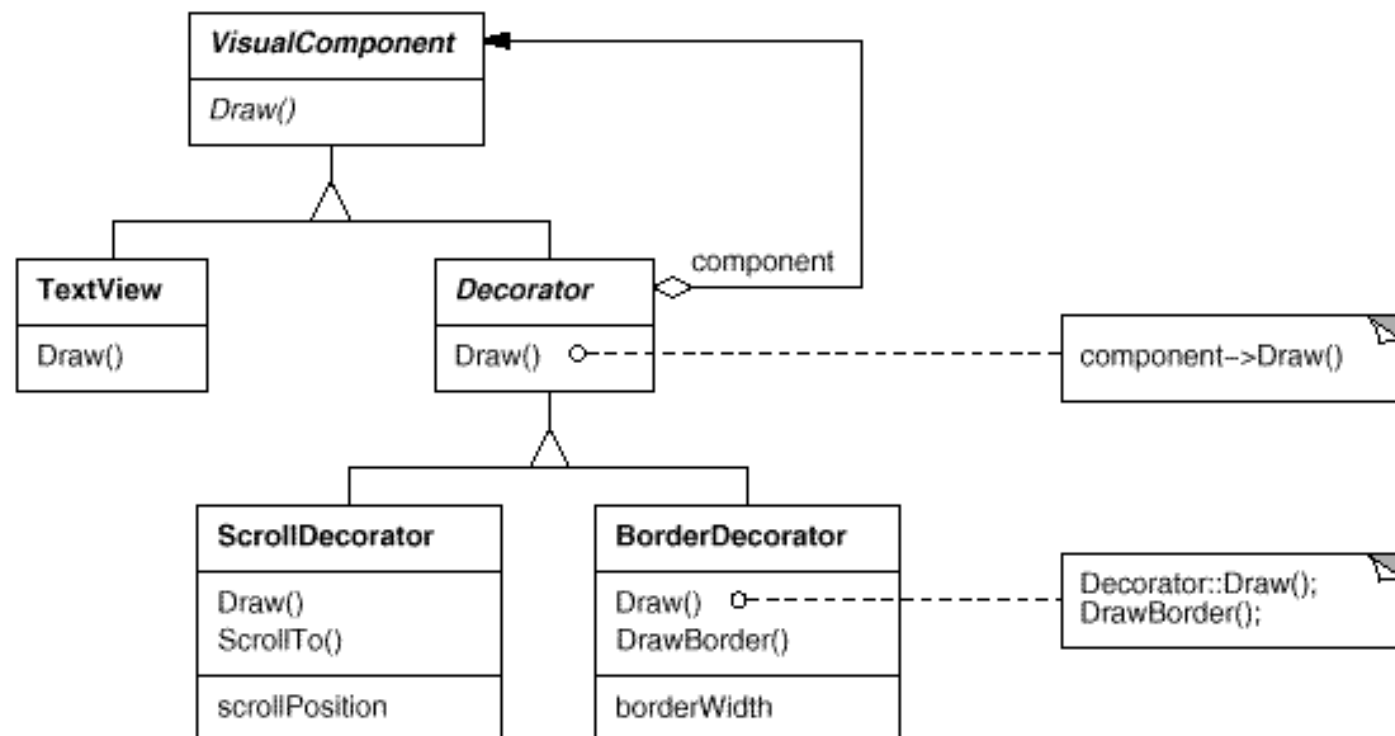


```
JPanel rootContainer = new JPanel();
JPanel topPanel = new JPanel();
rootContainer.add(topPanel, BorderLayout.NORTH);
topPanel.add(plus);
topPanel.add(minus);
label.setHorizontalAlignment(JLabel.CENTER);
rootContainer.add(label, BorderLayout.CENTER);
rootContainer.add(reset, BorderLayout.SOUTH);
```

Decorator

- Context
 - You have objects that implement a common interface, and you need to add responsibilities to individual objects dynamically.
- Problem
 - You want to extend an object's behaviour without modifying its class and without creating an explosion of subclasses for every combination of features

Decorator



Examples

- In Java
 - JScrollPane can be used to provide a scroll pane to any component

```
JScrollPane scrollPane=new JScrollPane(textArea);
```

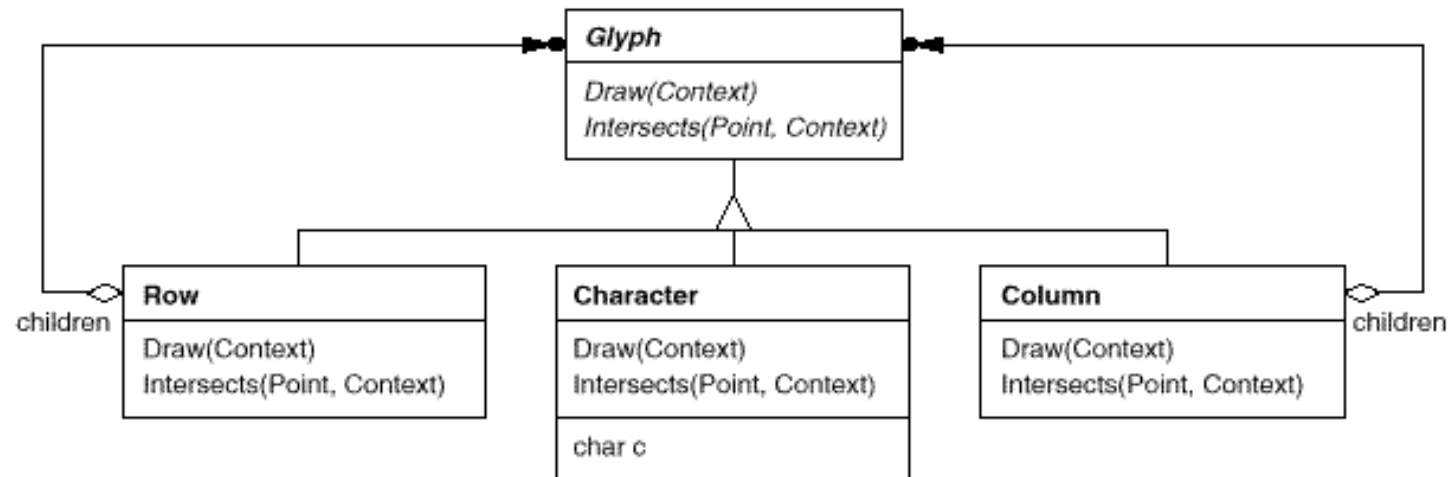
- BufferedReader provides buffering (method `readLine()`) to any reader

```
BufferedReader br = new BufferedReader(reader)
```

Flyweight

- Context
 - You need to create a very large number of similar objects, and memory usage becomes a concern.
- Problem
 - How can you support many fine-grained objects efficiently by sharing common (intrinsic) state, instead of duplicating it in every instance?

FlyWeight example



Flyweight example

- In Java the pooling of Integer objects is an example of FlyWeight
 - In this case there non extrinsic state

Proxy

- Context
 - You need to control access to an object.
- Problem
 - How can you provide a placeholder or surrogate for an object to manage access, defer creation, or add control logic, without changing the object's interface?

Proxy Variants

- A **remote proxy** provides a local representative for an object in a different address space.
- A **virtual proxy** creates expensive objects on demand. E.g. slow loading resource.
- A **protection proxy** controls access to the original object. E.g. when objects should have different access rights.
- A **smart reference** is a replacement for a bare pointer that performs additional actions when an object is accessed.
 - counting the number of references
 - loading a persistent object into memory
 - checking that the real object is locked before it's accessed

Proxy Example

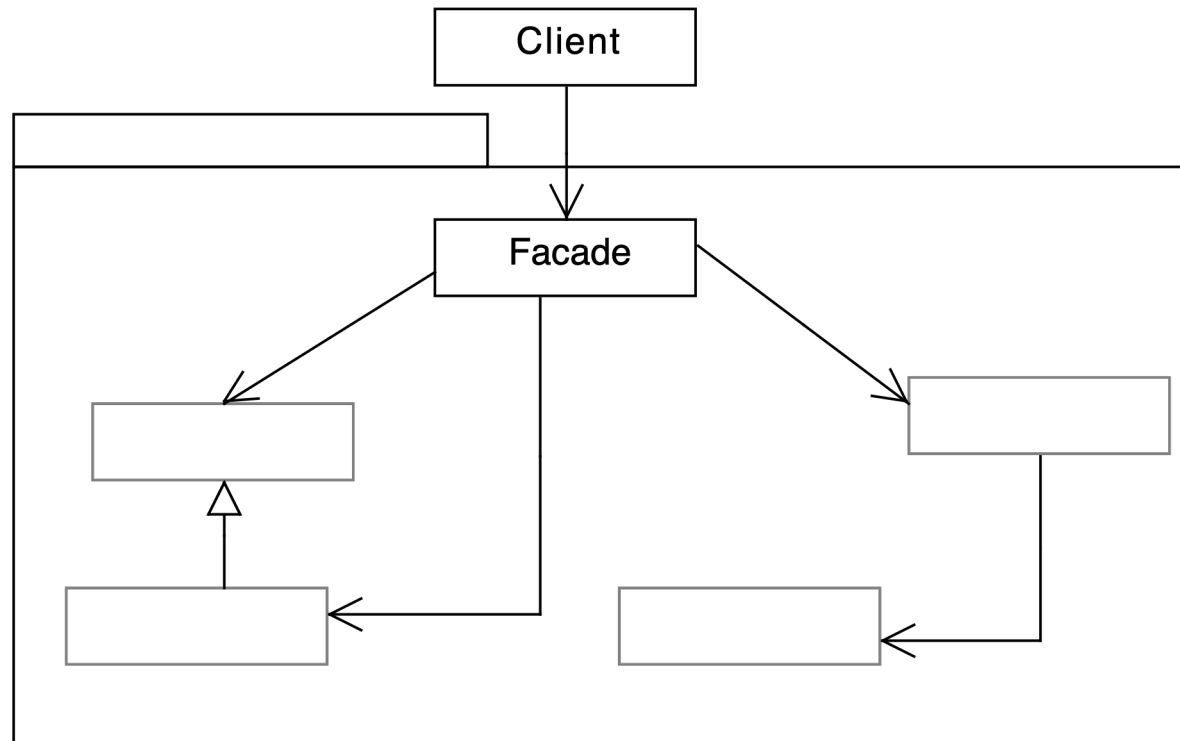
- The wrapper to make collections unmodifiable:

```
List<T> proxy = Collections.unmodifiableList(list);
```


Facade

- Context
 - A functionality is provided by a complex group of classes (interfaces, associations, etc.)
- Problem
 - How is it possible to use the classes without being exposed to the details

Facade



Behavioral

Design Patterns

Behavioral patterns

- Behavioral patterns are concerned with algorithms and the assignment of responsibilities between objects.
- Not just patterns of objects or classes but also the patterns of communication.
 - Complex control flow that's difficult to follow at run-time.
 - Shift focus away from flow of control to let concentrate just on the way objects are interconnected.

GoF behavioral patterns

- Object-level
 - Chain of Responsibility
 - Command
 - Iterator
 - Mediator
 - Memento
 - Observer
 - State
 - Strategy
 - Visitor
- Class-level
 - Template Method
 - Interpreter

Mechanisms

- Encapsulating variation
- Objects as arguments
- Information circulation policies
- Sender and Receiver decoupling

Encapsulating Variation

- A varying aspect of a program
- Captured by an object
 - Other delegate operations to the “variant” object
- Strategy
- State
- Mediator
- Iterator

Argument Objects

- Often an object is passed as argument
 - Hides complexity from clients
 - Concentrate the “active” code in one class
- Visitor
- Command
- Memento

Information circulation

- Responsibility of how to circulate information may be:
 - Distributed among different parties.
 - Encapsulated in a single object.

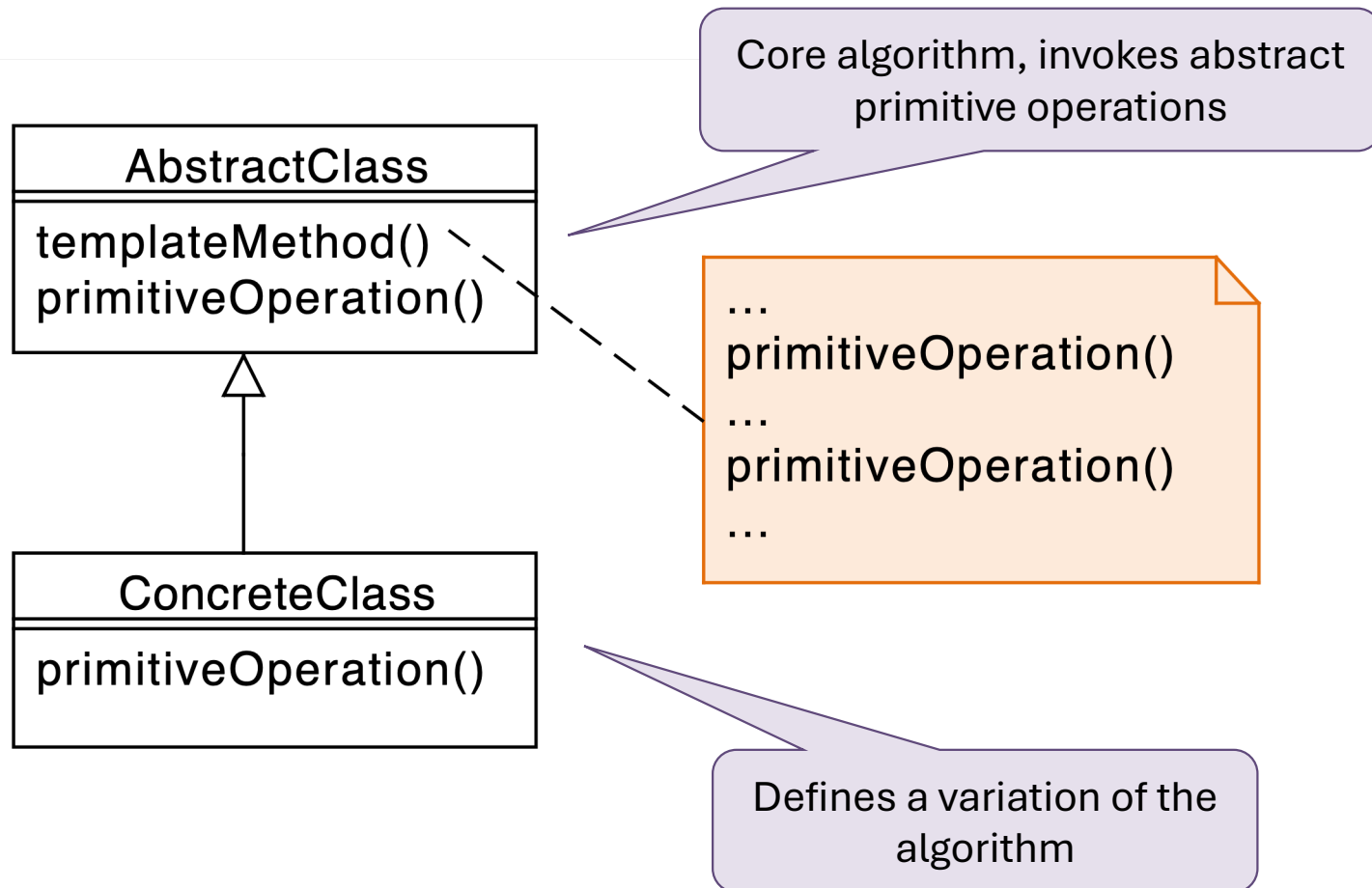
Communication decoupling

- Decoupling senders and receivers is a key to:
 - Reduce coupling
 - Improve reusability
 - Enforce layering and structure

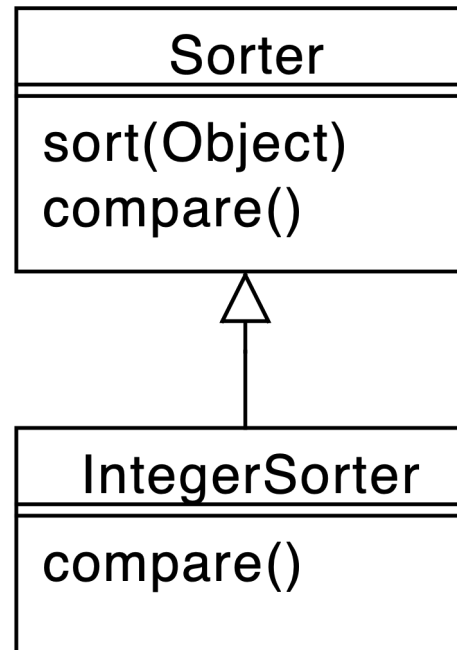
Template Method

- Context:
 - An algorithm/behavior has a stable core and several variation at given points
- Problem
 - You have to implement/maintain several almost identical pieces of code

Template Method



Template Method Example



Example: Sorter

```
public abstract class Sorter {  
    public void sort(Object v[]){  
        for(int i=1; i<v.length; ++i)  
            for(int j=0; j<v.length-i; ++j){  
                if(compare(v[j],v[j+1])>0){  
                    Object o=v[j];  
                    v[j]=v[j+1]; v[j+1]=o;  
                } } }  
    }  
    abstract int compare(Object a, Object b);  
}
```

Example: StringSorter

```
class StringSorter extends Sorter {  
    int compare(Object a, Object b){  
        String sa=(String)a;  
        String sb=(String)b;  
        return sa.compareTo(sb);  
    }  
}
```

```
Sorter ssrt = new StringSorter();  
String[] v={"g","t","h","n","j","k"};  
ssrt.sort(v);
```

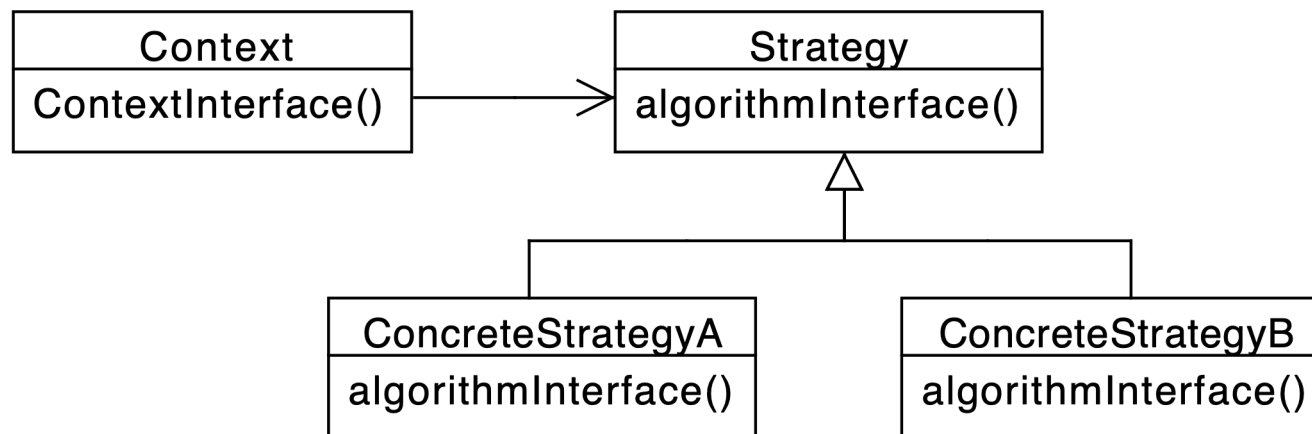
Strategy

- Context
 - Many classes or algorithm has a stable core and several behavioral variations
- Problem
 - Several different implementations are needed.
 - Multiple conditional constructs tangle the code.

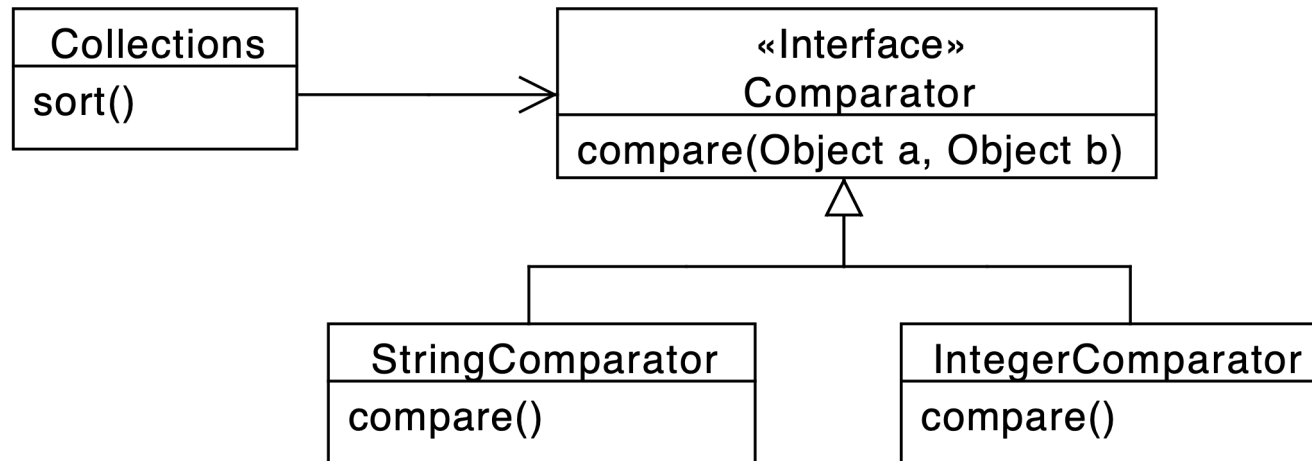
Strategy Pattern

- Solution
 - Embed each variation inside a strategy object passed as a parameter to the algorithm
 - The strategy object's class implements an interface providing the operations required by the algorithm

Strategy



Strategy example: Comparator



Comparator

- Interface java.util.Comparator

```
public interface Comparator<T>{  
    int compare(T a, T b);  
}
```

- Semantics (as comparable): returns
 - a negative integer if a precedes b
 - 0, if a equals b
 - a positive integer if a succeeds b

Comparator

```
class StudentCmp
    implements Comparator<Student>{
public int compare(Student a, Student b){
    return a.id - b.id;
}
}
```

```
Student[] sv = { new Student(11),
                  new Student(3),
                  new Student(7)};
Arrays.sort(sv, new StudentCmp());
```

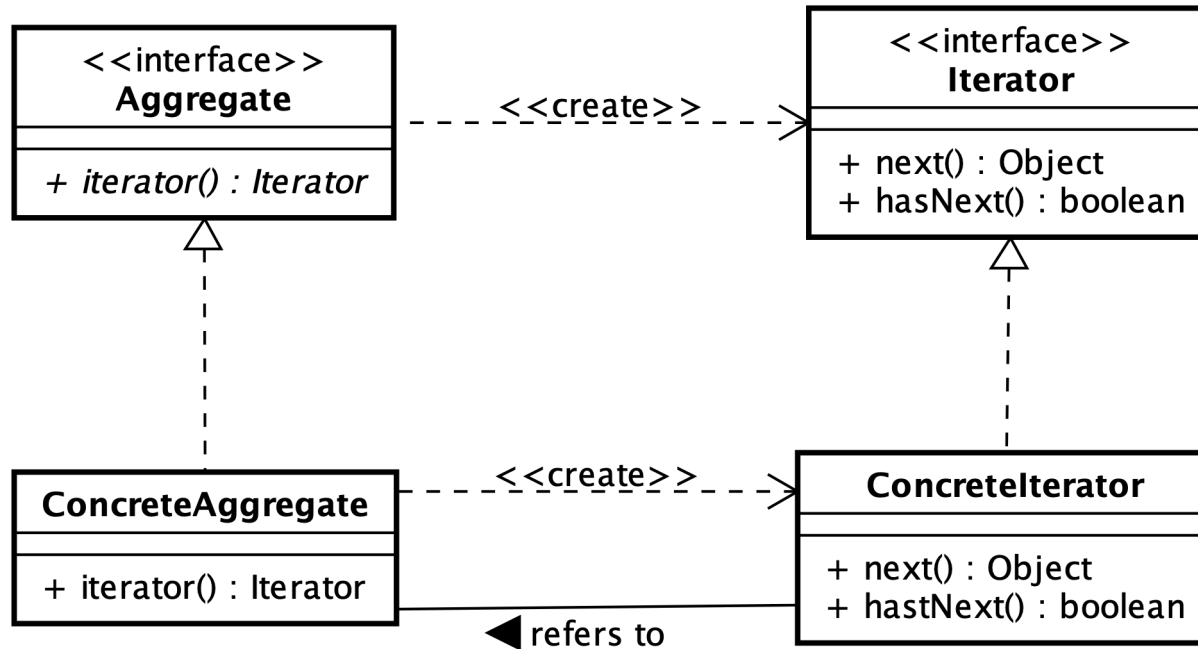
Strategy Consequences

- + Avoid conditional statements
- + Algorithms may be organized in families
- + Choice of implementations
- + Run-time binding
- Clients must be aware of different strategies
- Communication overhead
- Increased number of objects

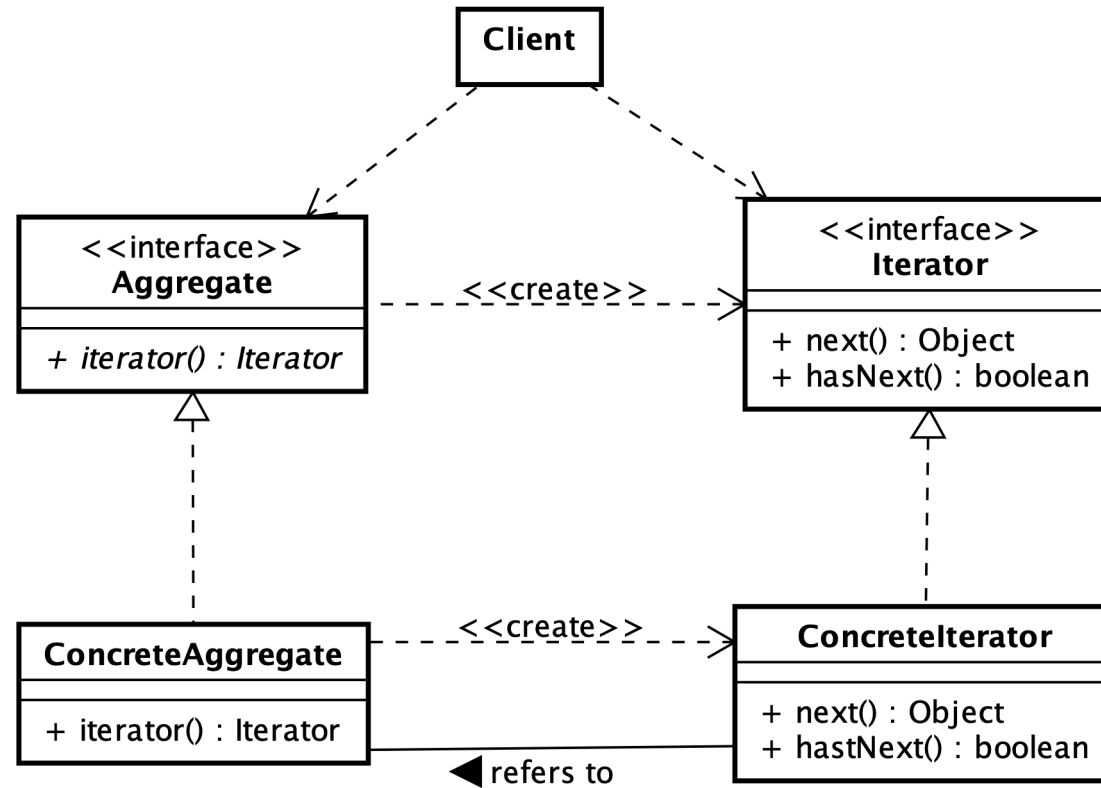
Iterator pattern

- Context
 - A collection of objects must be iterated
- Problem
 - Multiple concurrent iterations are possible
 - The internal storage must not be exposed
- Solution
 - Provide an iterator object, attached to the collection, that can be advanced independently

Iterator pattern



Iterator pattern



Common behavior: iteration

- Interface `java.lang.Iterable`

```
public interface Iterable<T> {  
    Iterator<T> iterator();  
}
```

- Any class implementing `Iterable` can be the target of a for-each construct
 - Uses the `Iterator` interface

Common behavior: iteration

- Interface java.util.Iterator

```
public interface Iterator<T> {  
    boolean hasNext();  
    T next();  
}
```

- Semantics:
 - Initially positioned before the first element
 - `hasNext()` tells if a next element is present
 - `next()` returns the next element and advances by one position

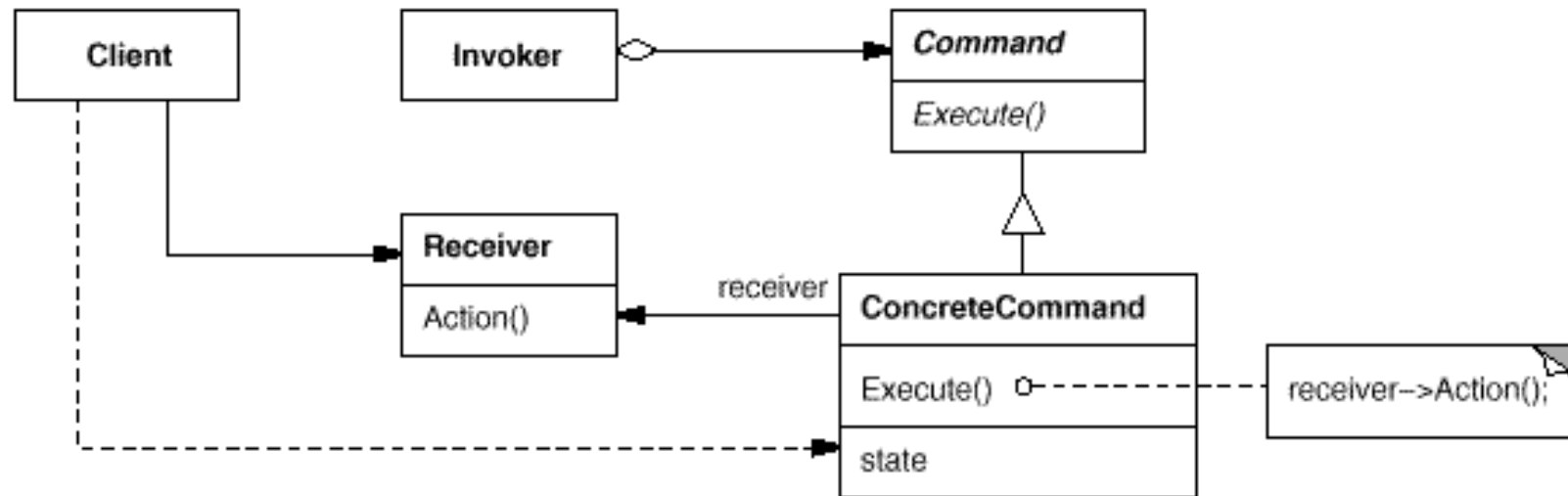
Iterable example

```
class Random implements Iterable<Integer> {  
    private int[] values;  
    public Random(int n, int min, int max){ ... }  
    public Iterator iterator() {  
        return new Iterator<Integer> {  
            private int next=0;  
            public boolean hasNext() {  
                return next < values.length; }  
            public Integer next() {  
                return new Integer(values[next++]);}  
        }; }  
}
```

Command

- Context
 - You need to issue requests (operations) to objects, possibly queuing, logging, undoing, or parameterizing them.
- Problem
 - How can you encapsulate a request as an object so that you can decouple the sender from the receiver and treat operations uniformly (store, queue, undo, or execute later)?

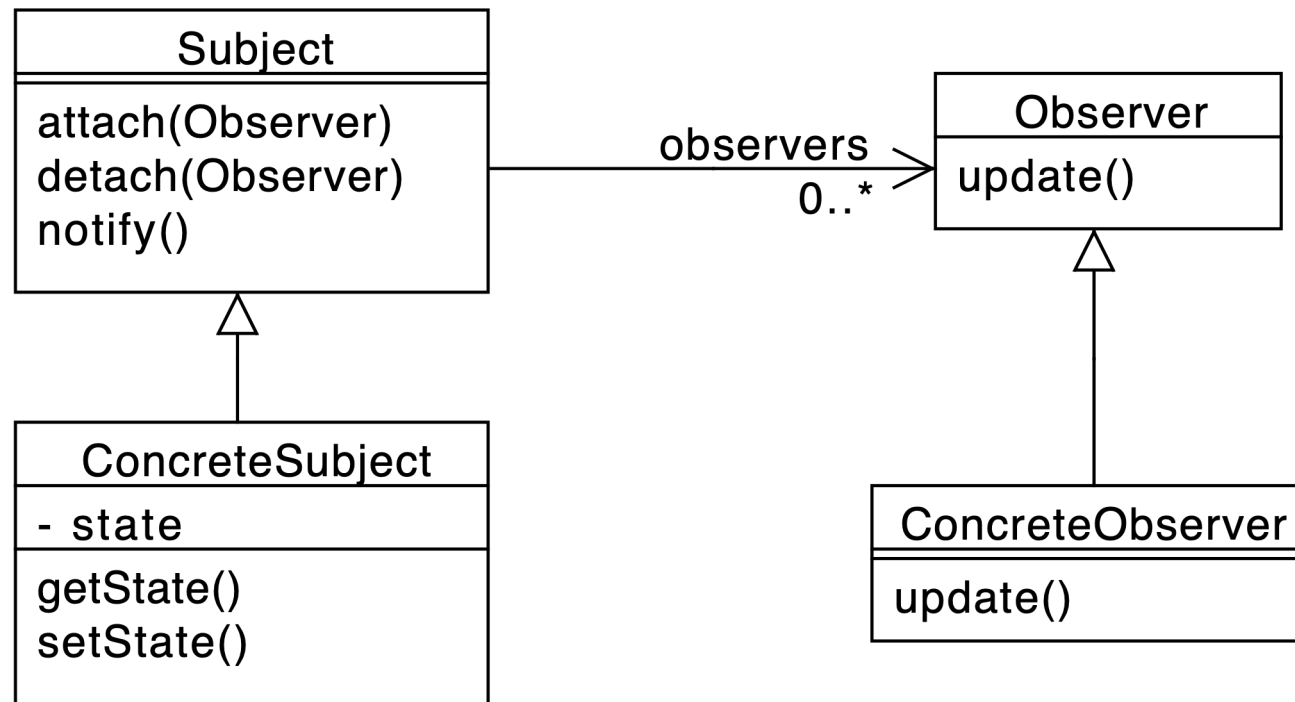
Command Pattern



Observer

- Context:
 - The change in one object may influence one or more other objects
- Problem
 - High coupling
 - Number and type of objects to be notified may not be known in advance

Observer



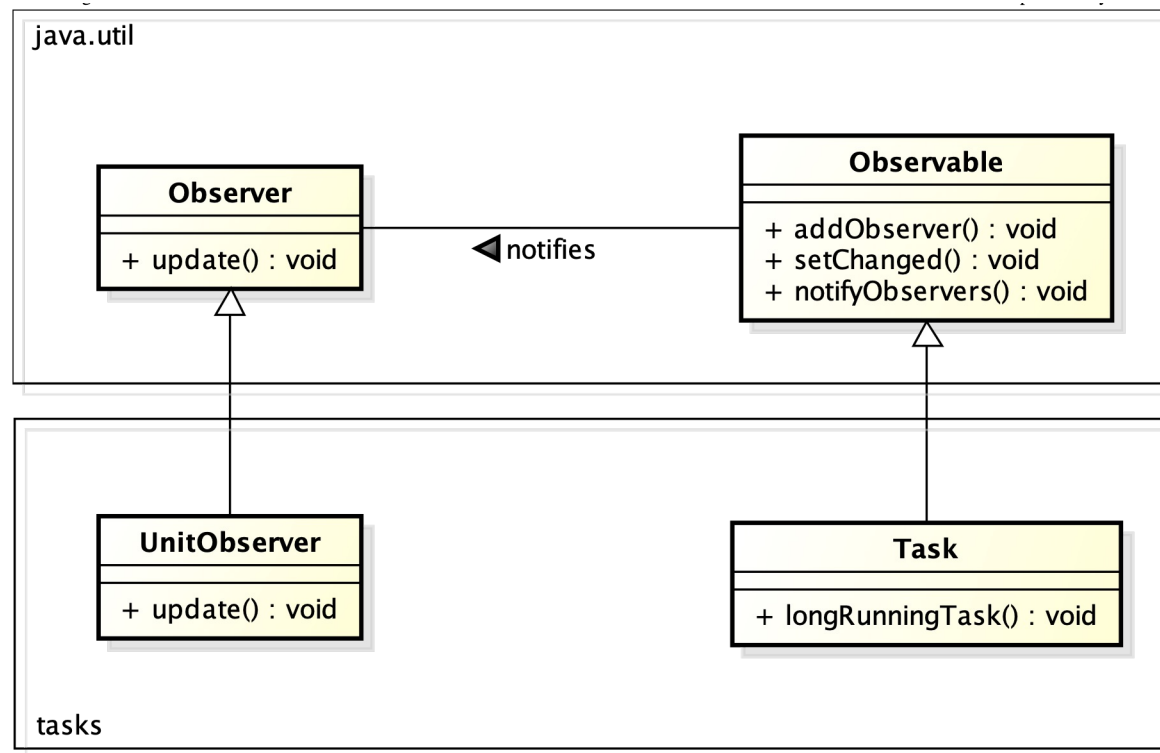
Observer – Consequences

- + Abstract coupling between Subject and Observer
- + Support for broadcast communication
- Unanticipated updates

Java Observer-Observable

- Allow a standardized interaction between an objects that needs to notify one or more other objects
- Defined in package `java.util`
- Class `Observable`
- Interface `Observer`

Observer-Observable



Java Observer-Observable

```
class Observable{  
    void addObserver( .. ){}  
    void deleteObserver( .. ){}  
    void deleteObservers(){}  
    int countObservers() {}  
    void setChanged() {}  
    void clearChanged() {}  
    boolean hasChanged() {}  
    void notifyObservers() {}  
    void notifyObservers( .. ) {}  
}
```

```
interface Observer  
    void update(Observable o,  
                Object arg);  
}
```

Observer-Observable

- Class `Observable` manages:
 - registration of interested observers by means of method `addObserver()`
 - sending the notification of the status change to the observer(s) together with additional information concerning the status (event object).
- Interface `Observer` allows:
 - Receiving standardized notification of the observer change of state through method `update()` accepts two arguments:
 - Observable object that originated the notification
 - additional information (the event object)

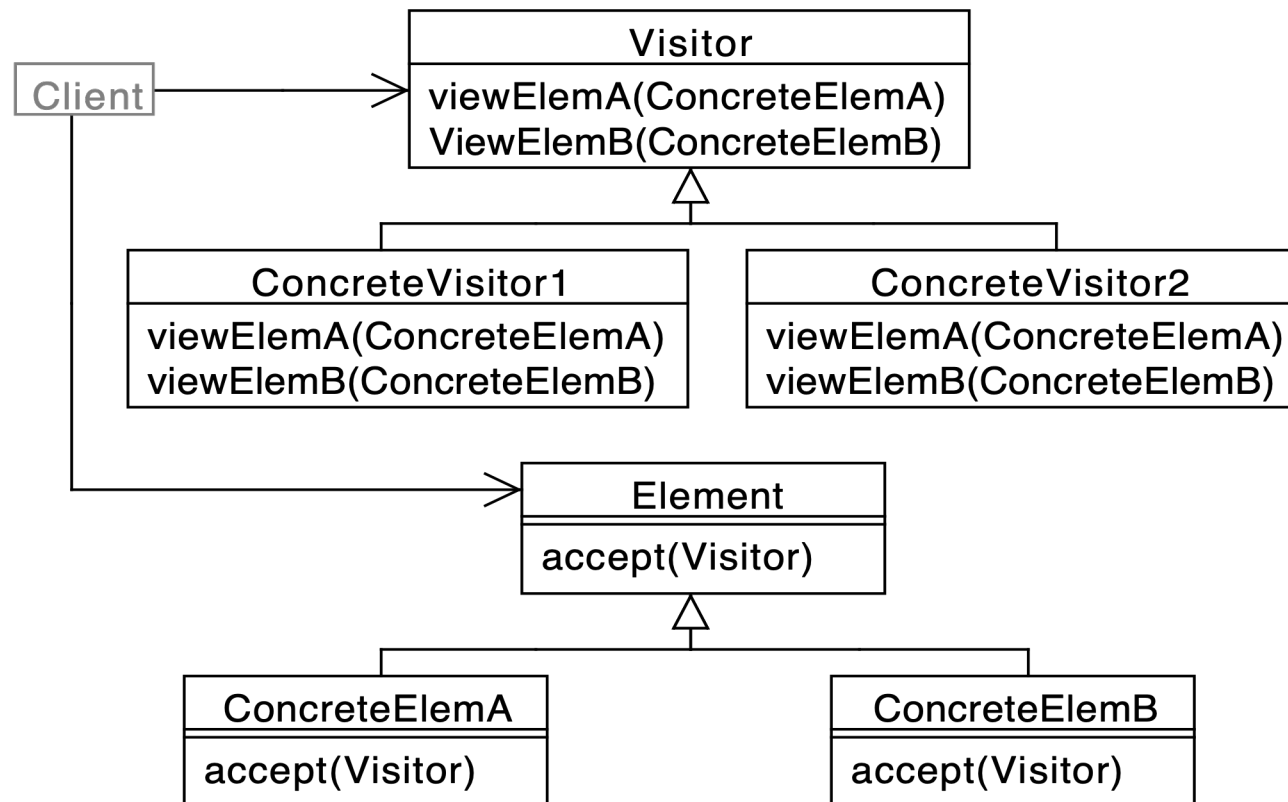
Observer-Observable

- Sending a notification from an observable element involves two steps:
 - record the fact the the status of the Observable has changed, by means of method `setChanged()`,
 - send the actual notification while providing the additional information (the event object), by means of method `notifyObservers()`

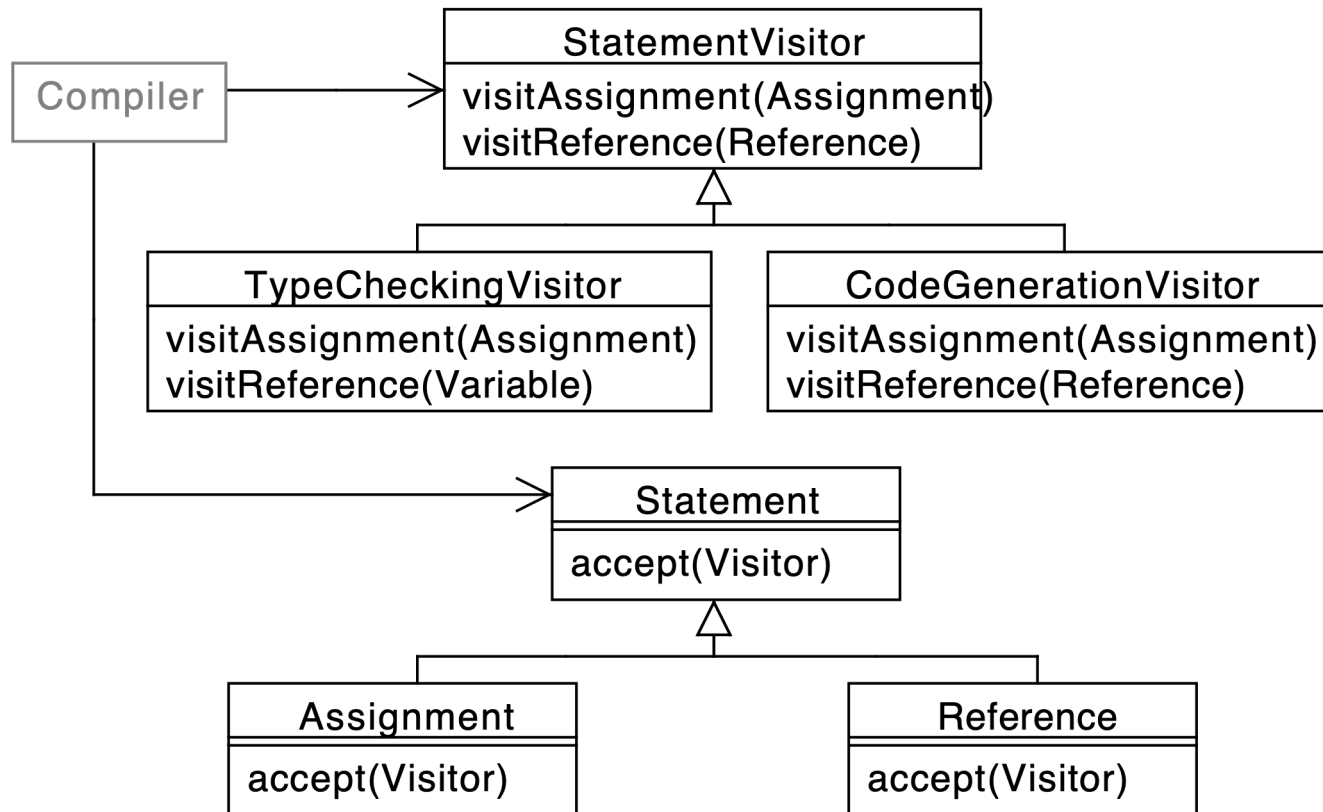
Visitor

- Context
 - An object structure contains many classes with differing interfaces.
 - Many different operations need to be performed on the objects
- Problem
 - The operations on the objects depend on their concrete classes
 - Classes could be polluted with several operations

Visitor



Visitor Example



Visitor Consequences

- + Adding new operations is very easy
- + Behavior is partitioned
- + Can visit class hierarchies
- + State can be accumulated

- Difficult to add new concrete elements
- Break of encapsulation

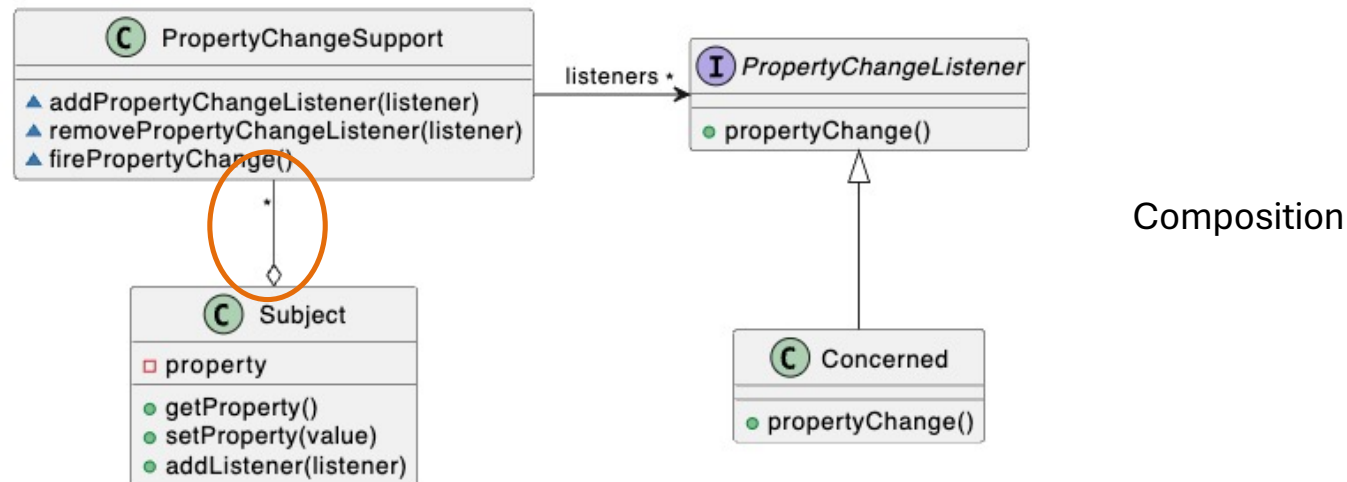
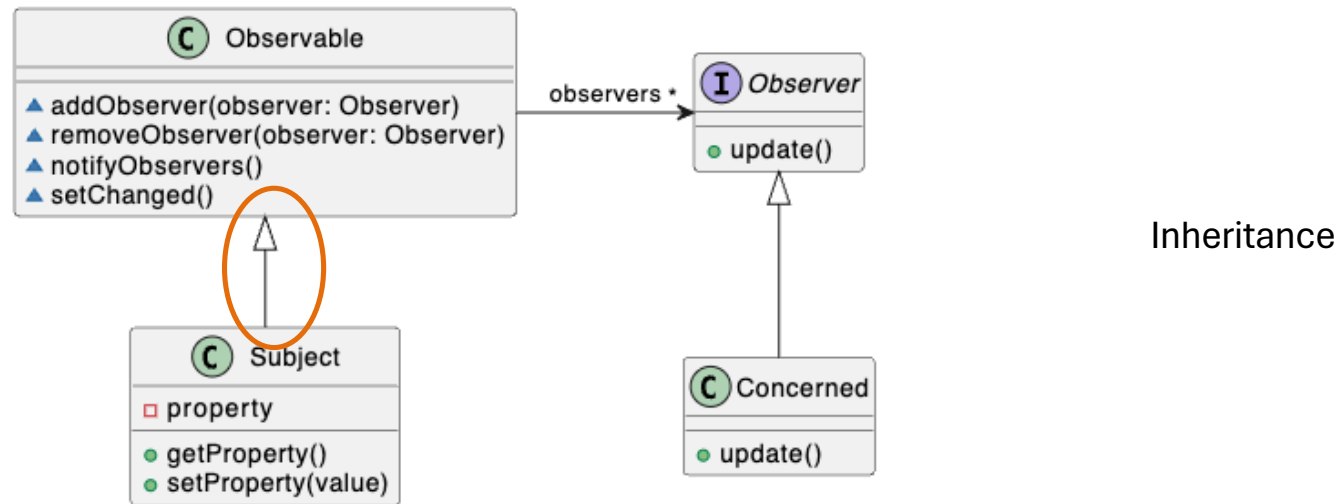
Inheritance vs. Composition

Inheritance vs. composition

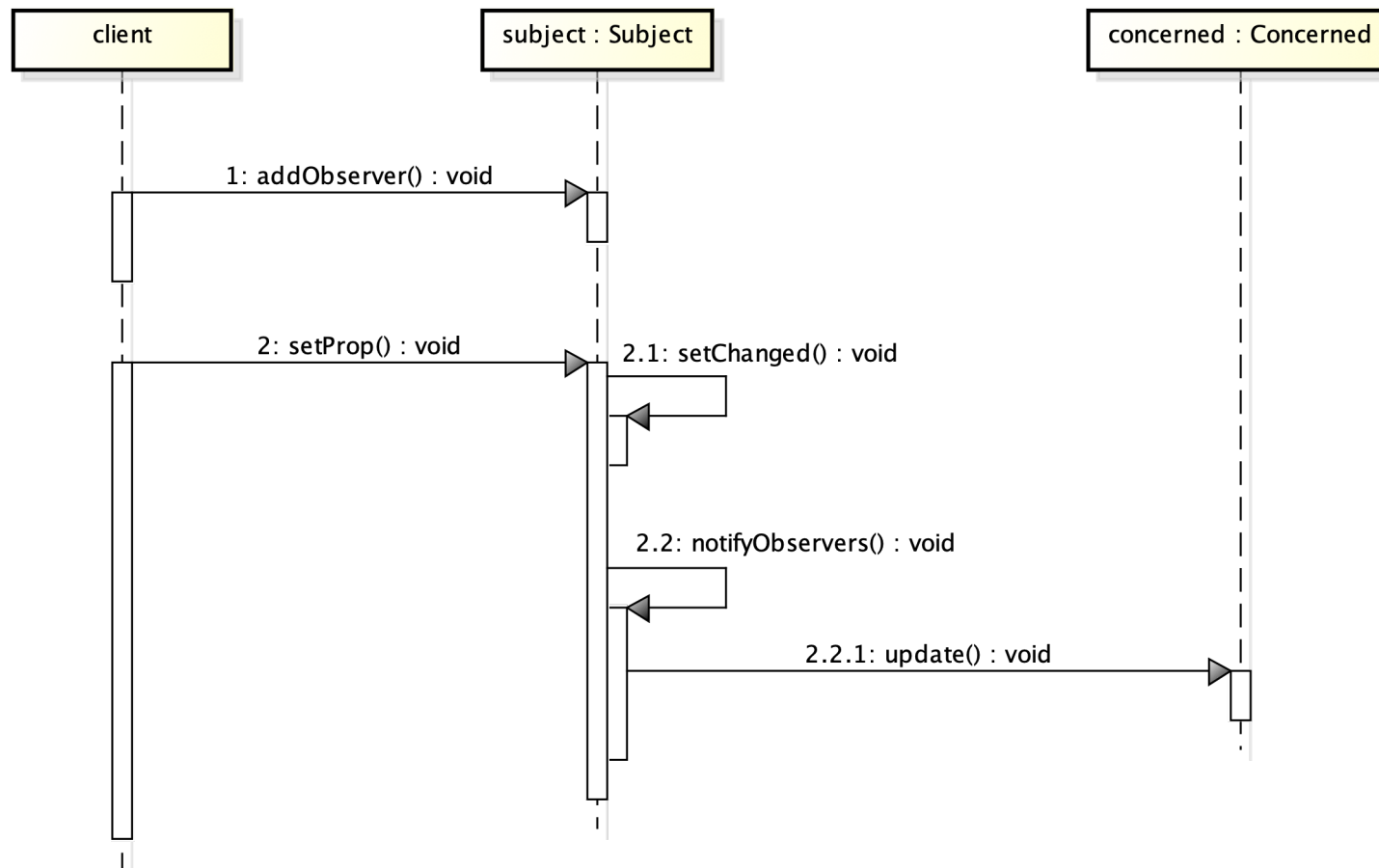
Reuse can be achieved via:

- Inheritance
 - The reusing class has the reused methods available as own methods.
 - Clients can invoke directly inherited methods
- Composition
 - The reusing class has the reused methods available in an included object (attribute)
 - The reusing class must provide methods that accept clients requests and delegate to the included object

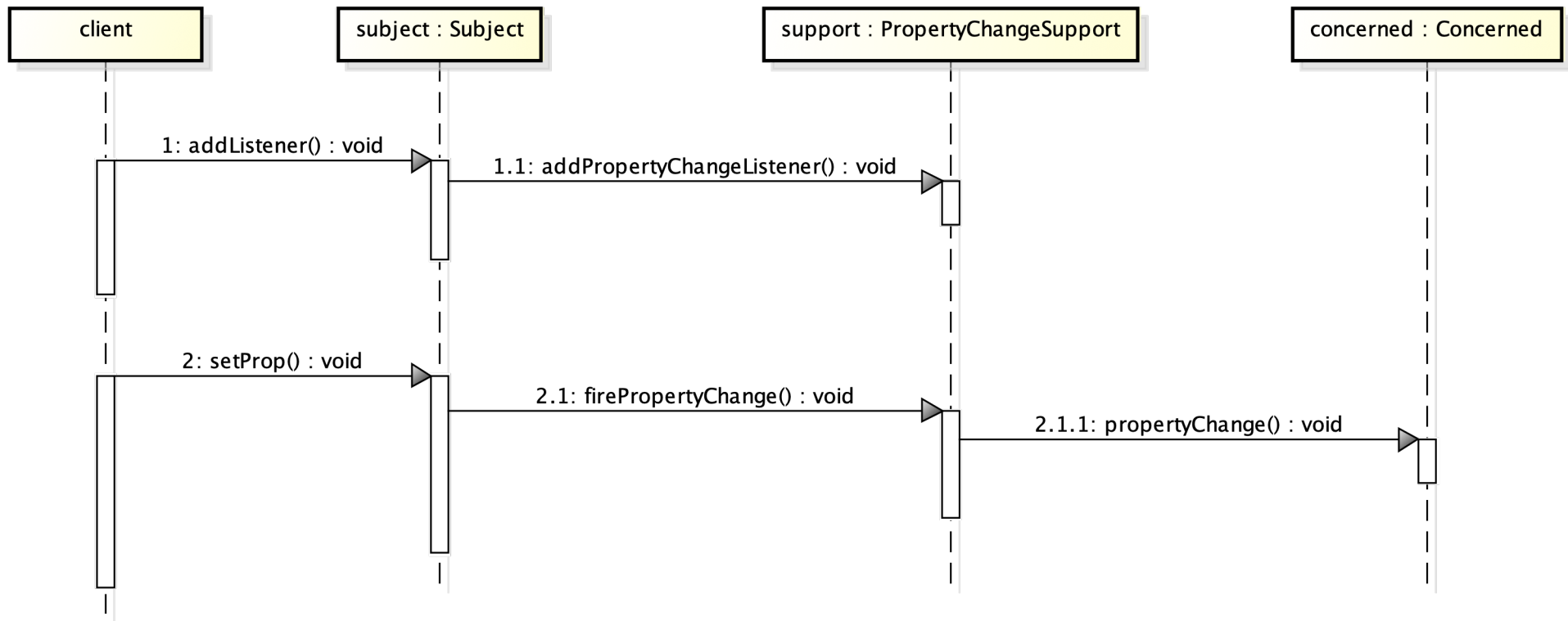
Inheritance vs. Composition



Observer w/Inheritance



Observer w/Composition



Observable subject w/inheritance

```
public class Subject
    extends Observable {

    String prop="ini";
    public void setProp(String val){
        setChanged();
        property = val;
        notifyObservers("theProp");
    }
}
```


Observable subject w/composition

```
public class Subject {  
    PropertyChangeSupport pcs =  
        new PropertyChangeSupport(this);  
    String prop="ini";  
    public void setProp(String val) {  
        String old = property;  
        property = val;  
        pcs.firePropertyChange("theProp", old, val);  
    }  
    public void addObs(PropertyChangeListener l){  
        pcs.addPropertyChangeListener("theProp",l);  
    } }  
}
```

Observer w/inheritance

```
public class Concerned
    implements Observer {
    @Override
    public void update(Observable src, Object arg) {
        System.out.println("Variation of " + arg);
    }
}
```

Observer w/composition

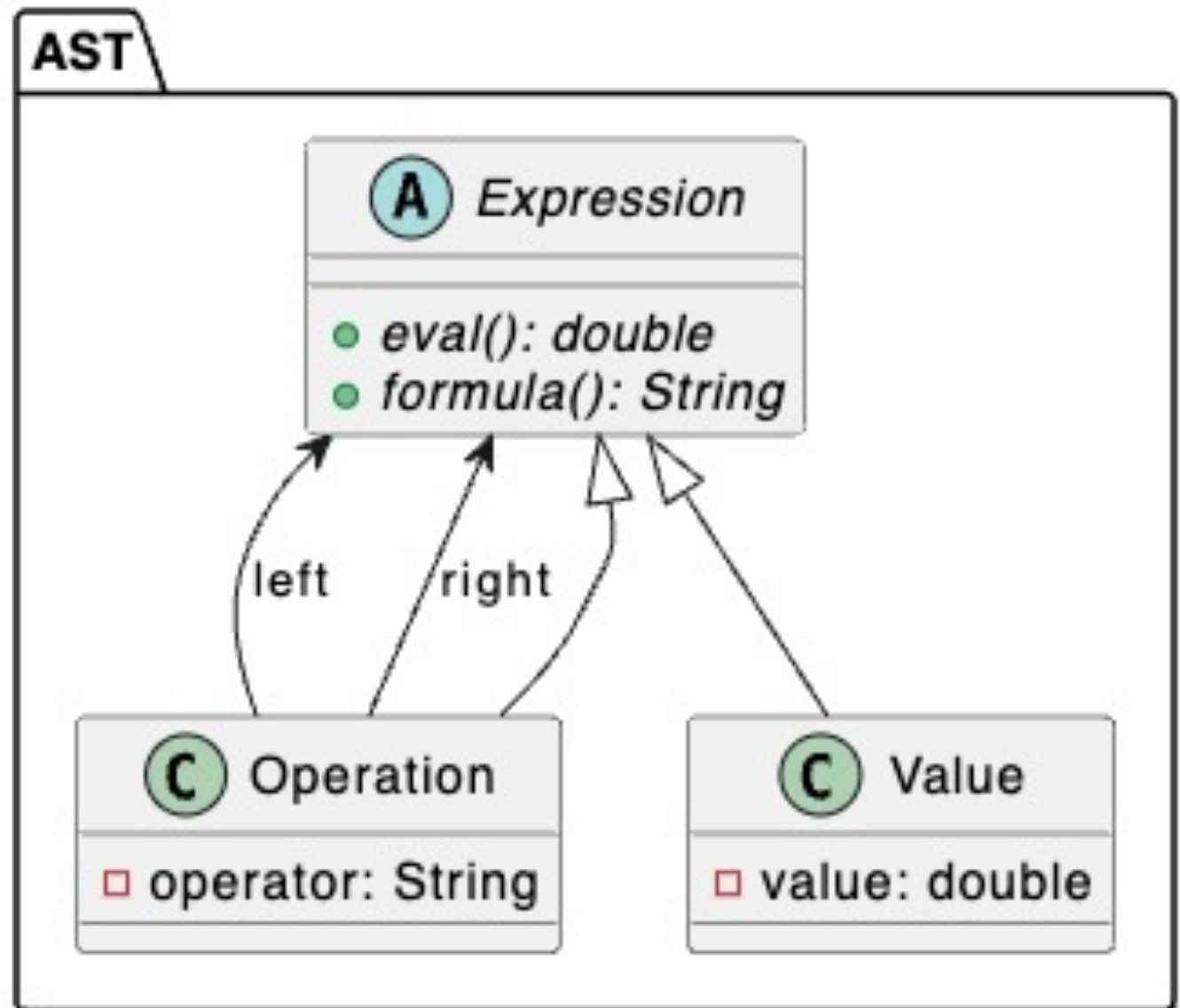
```
public class Concerned
    implements PropertyChangeListener {
    @Override
    public void propertyChange(PropertyChangeEvent evt) {
        System.out.println("Variation of " +
            evt.getPropertyName());
    }
}
```

Abstract Syntax Tree

Abstract Syntax Tree

- Arithmetic expressions representation
 - Operators
 - Operands
 - Composite Pattern
- Evaluation of expressions
 - Interpreter Pattern
 - Visitor Pattern

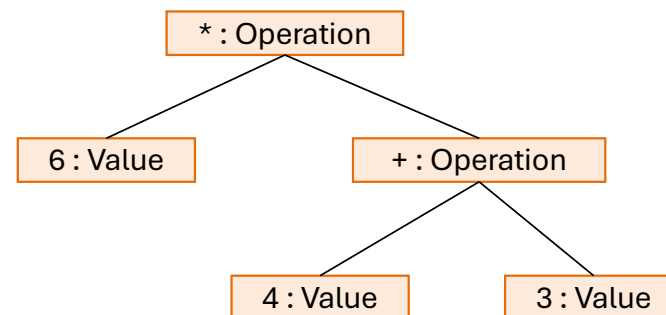
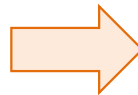
Interpreter



Expression Tree example

```
Expression e =  
    new Operation('*',  
        new Value(6),  
        new Operation('+',  
            new Value(4),  
            new Value(3)))  
IO.println( e.formula() + " = " + e.eval());
```

$6 * (4 + 3)$



Expression Tree example

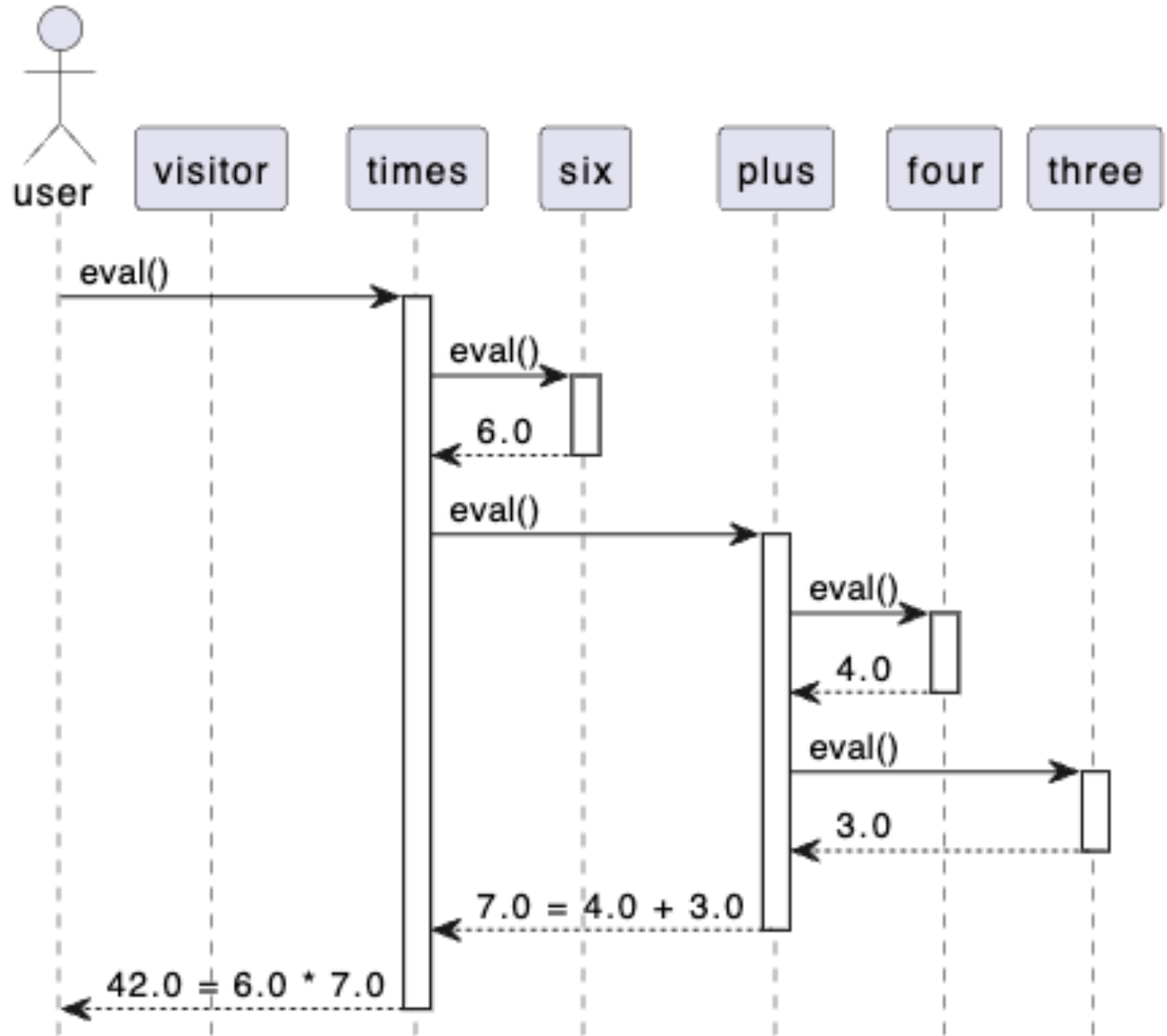
```
public abstract class Expression{  
    public abstract double eval();  
    public abstract String formula();  
}
```

```
public class Value extends Expression {  
    private double value;  
    public Value(double v){ value = v; }  
    @Override  
    public double eval() { return value;  
    }  
    @Override  
    public String formula() {  
        return String.valueOf(value);  
    }  
}
```

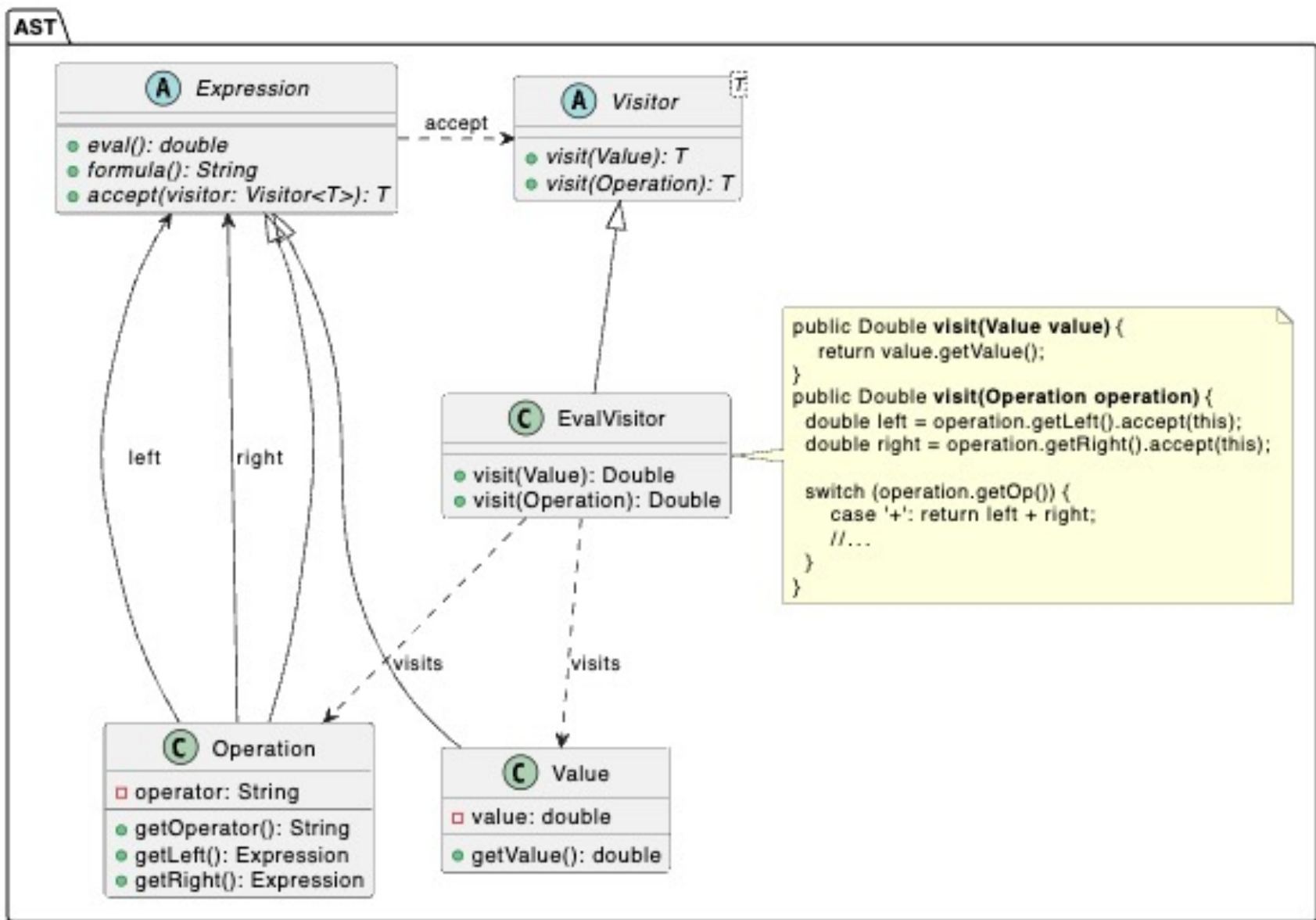

Expression Tree example

```
public class Operation extends Expression {
    private char op;
    private Expression left, right;
    public Operation(char o, Expression l, Expression r){
        op = o; left=l; right=r;
    }
    public double eval() {
        switch(op){
            case '+': return left.eval() + right.eval();
            ... }
        }
    public String formula() {
        return "("+left.formula()+" "+op+" "+right.formula()+")";
    } }
}
```

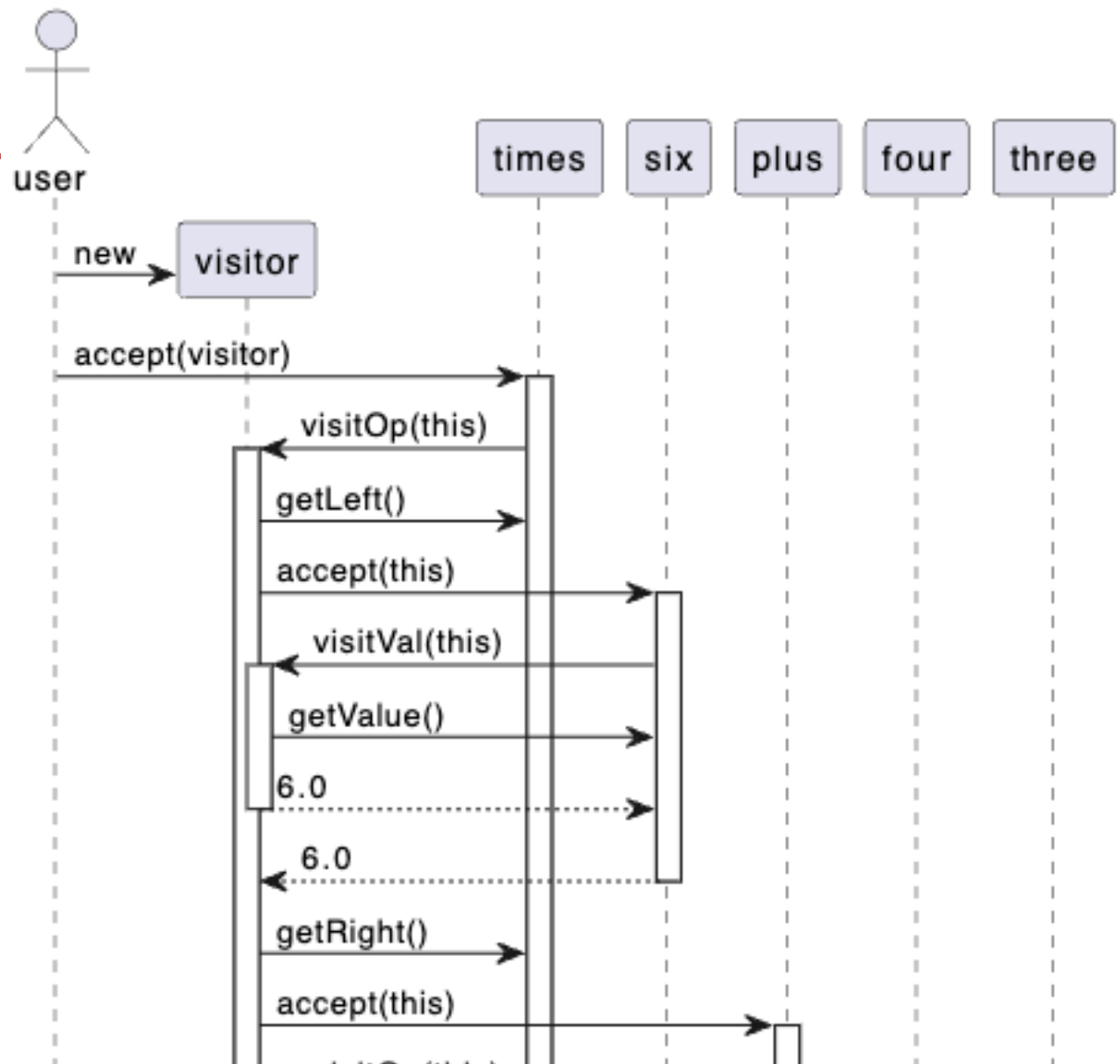
Expression Tree



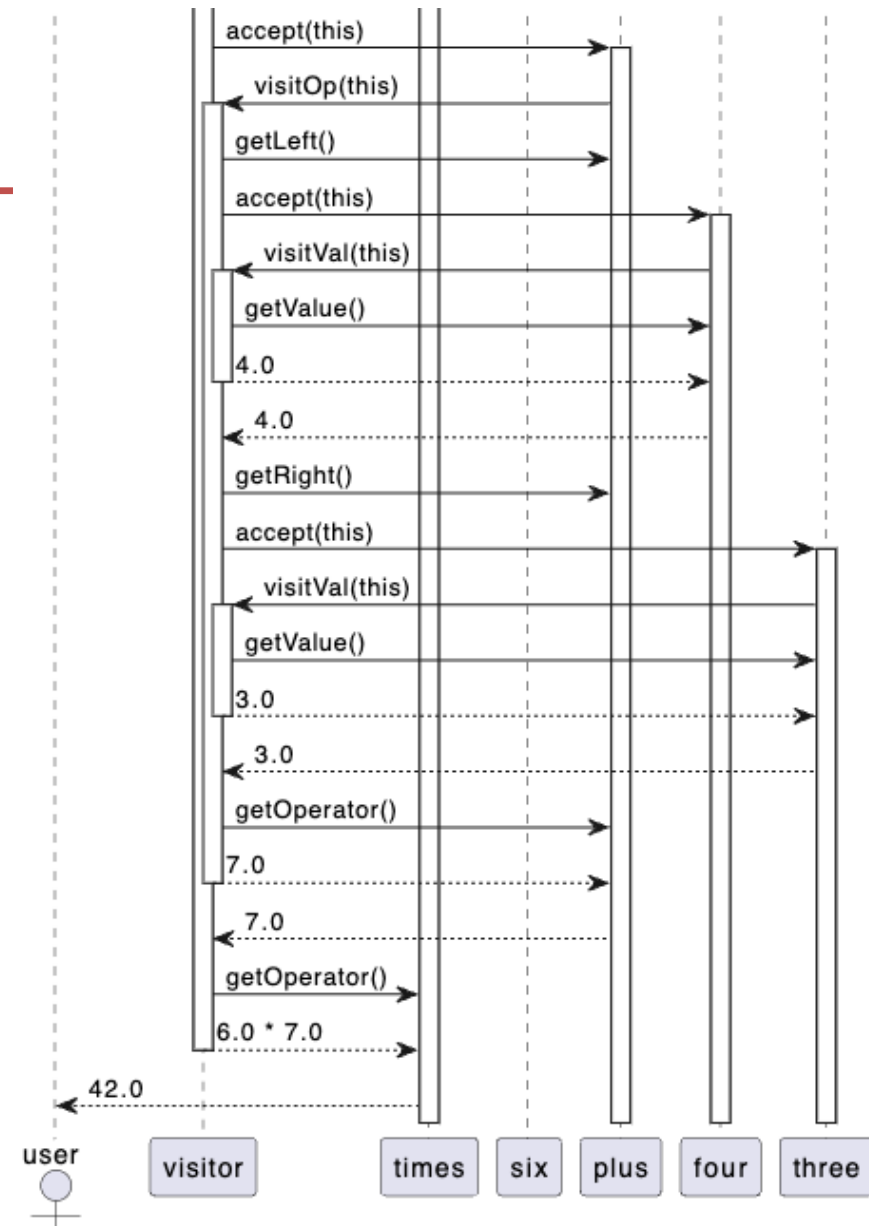
Visitor



Visitor



Visitor

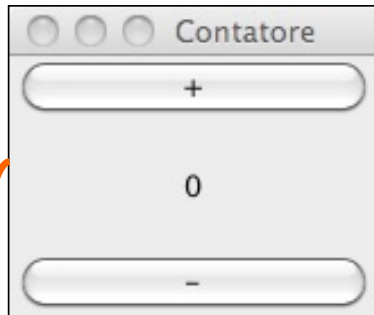


Model-View-Controller

MVC

- Context
 - Interactive applications with flexible HCI
- Problem
 - The same information is presented in different ways/windows
 - Windows must present consistent data
 - Should be possible to add/change presentations of information

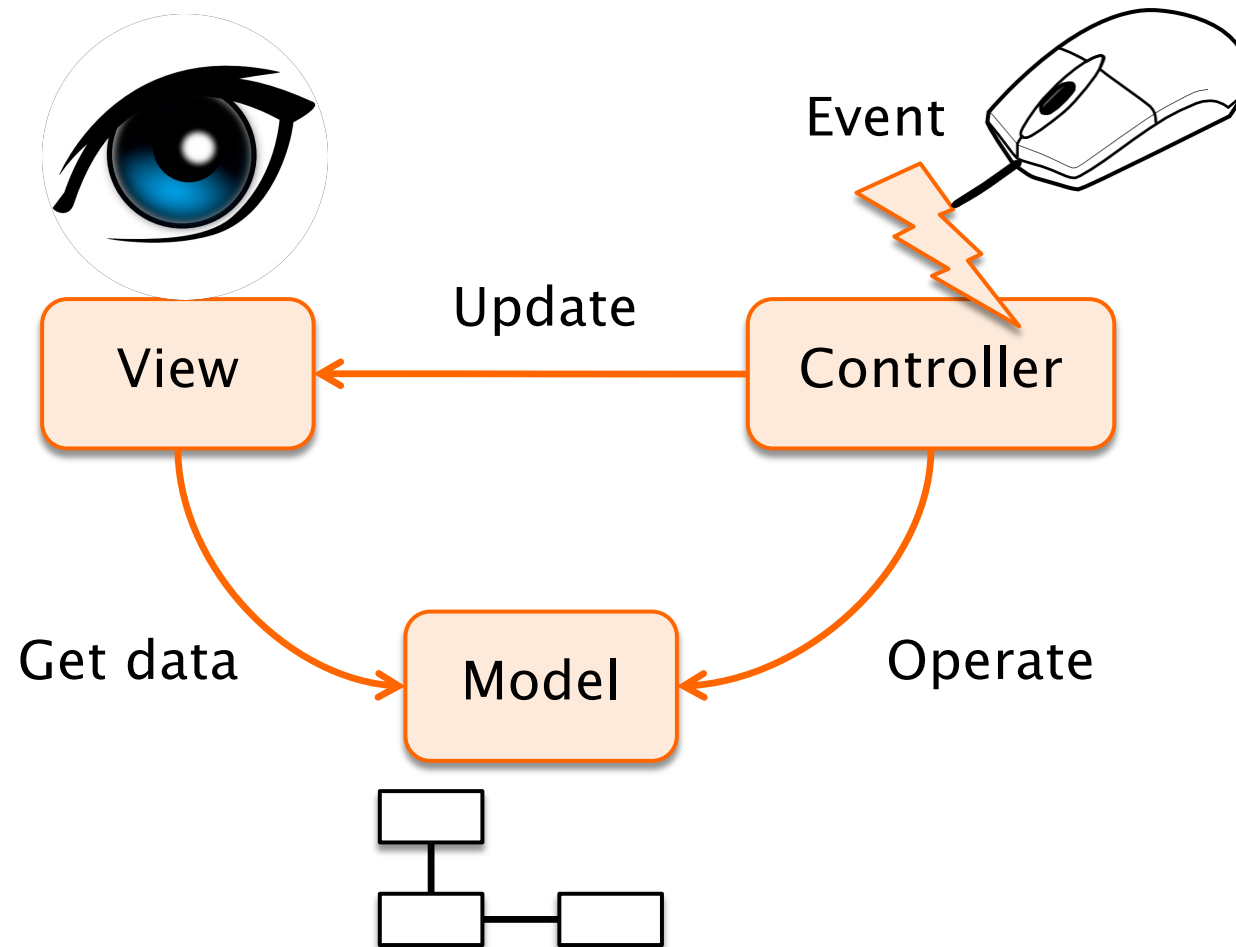
MVC example



```
plusBtnClick() {  
    theCounter.increment();  
    view.update();  
}  
  
minusBtnClick() {  
    theCounter.decrement();  
    view.update();  
}
```

```
class Counter{  
    private int value;  
    public void increment(){ value++;}  
    public void decrement(){ value--;}  
    public int getValue(){ return value;}  
}
```


MVC



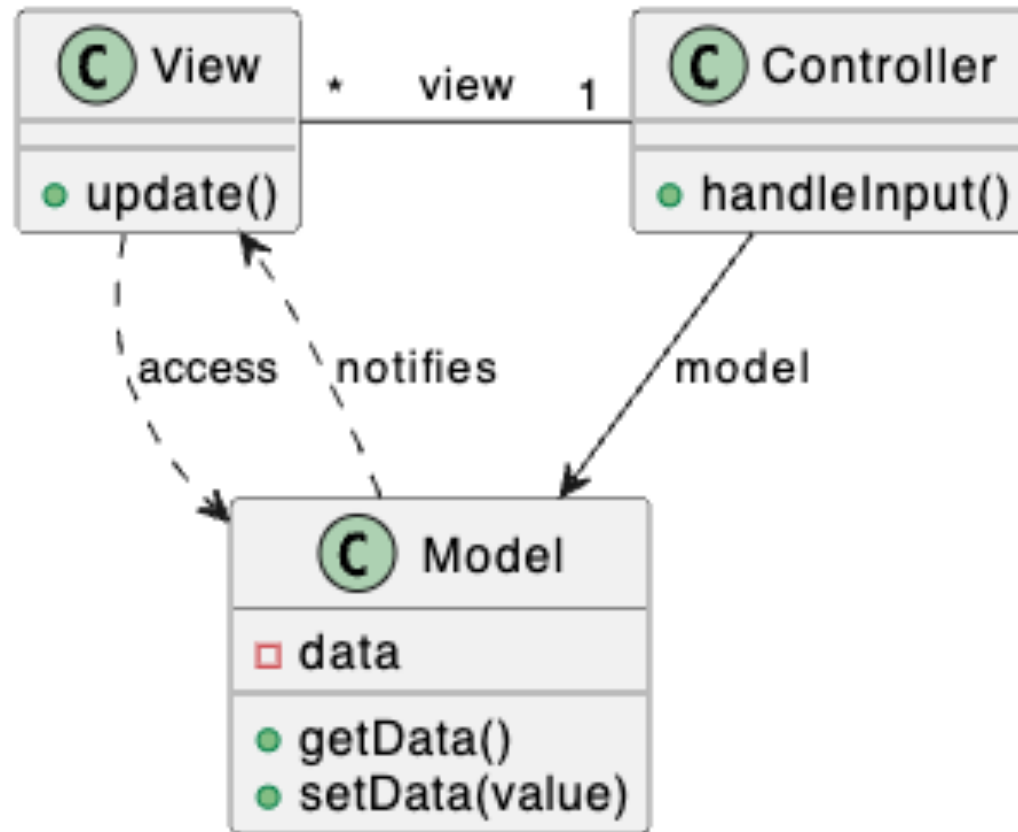
MVC Principles

- When building a GUI we must consider two main aspects:
 - Layout (View): how to place the graphical elements to achieve a give visual aspect
 - Events (Controller): which behavior associate to elements' events
- Application logic (Model) must remain, as far as possible, separate from user interface.

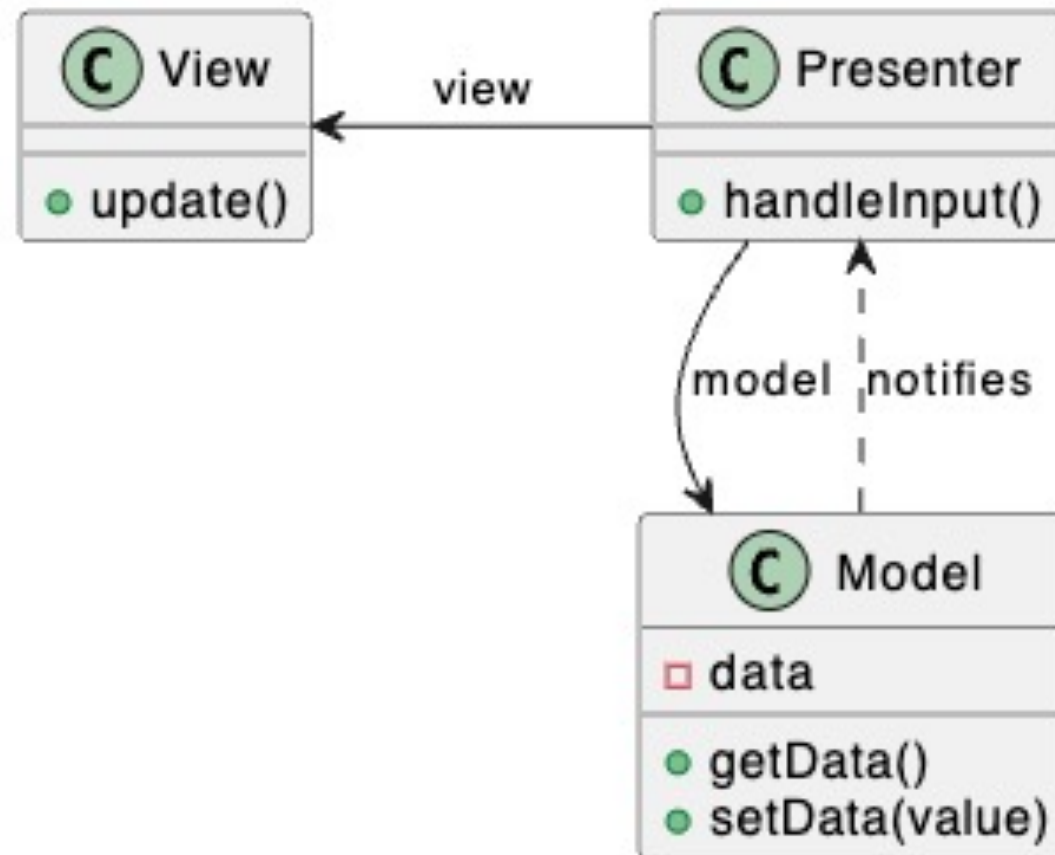
MVC Principles

- Layout (View): how to place the graphical elements to achieve a give visual aspect
- Events (Controller): which behavior associate to elements' events
- Application logic (Model) must remain, as far as possible, separate from user interface

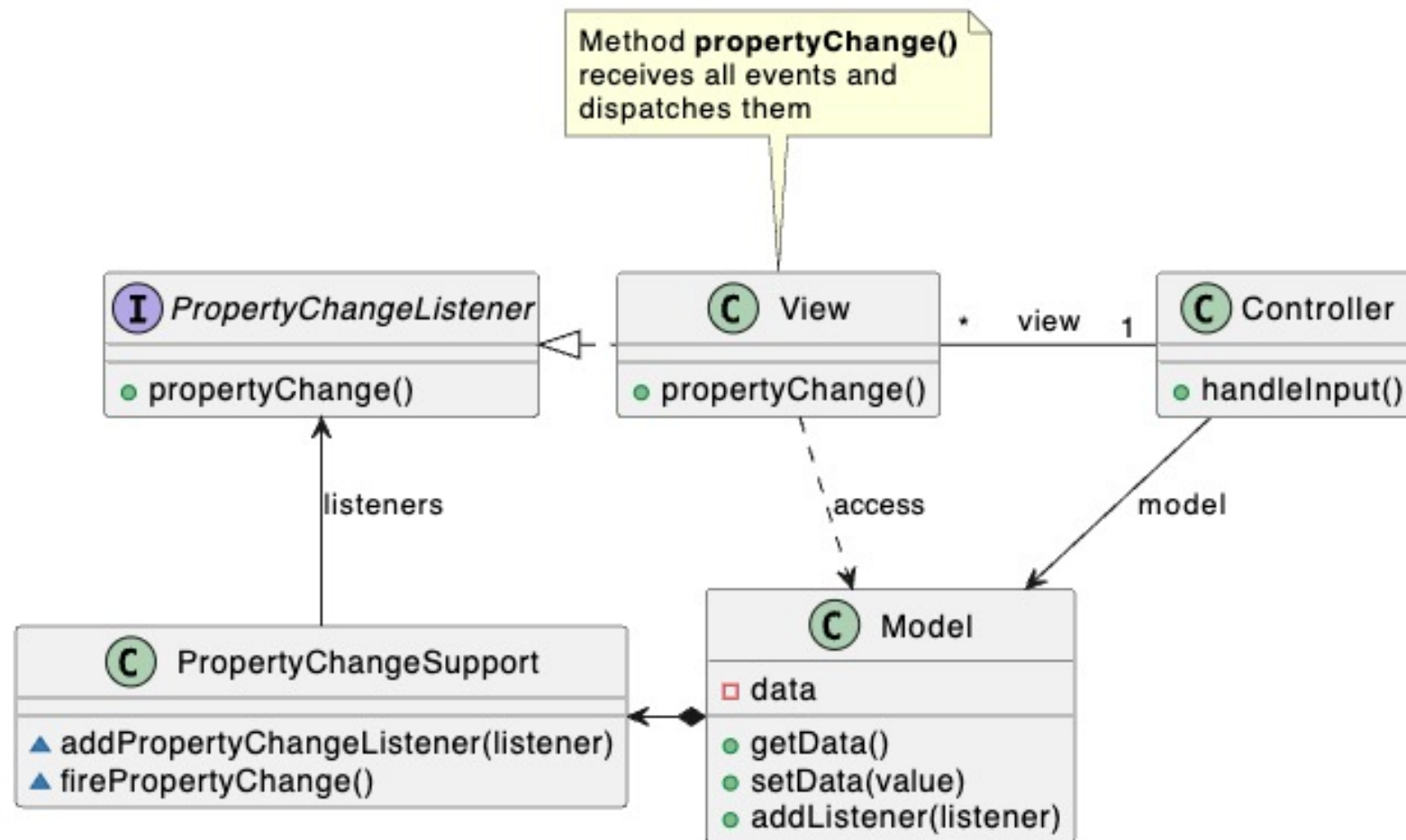
MVC



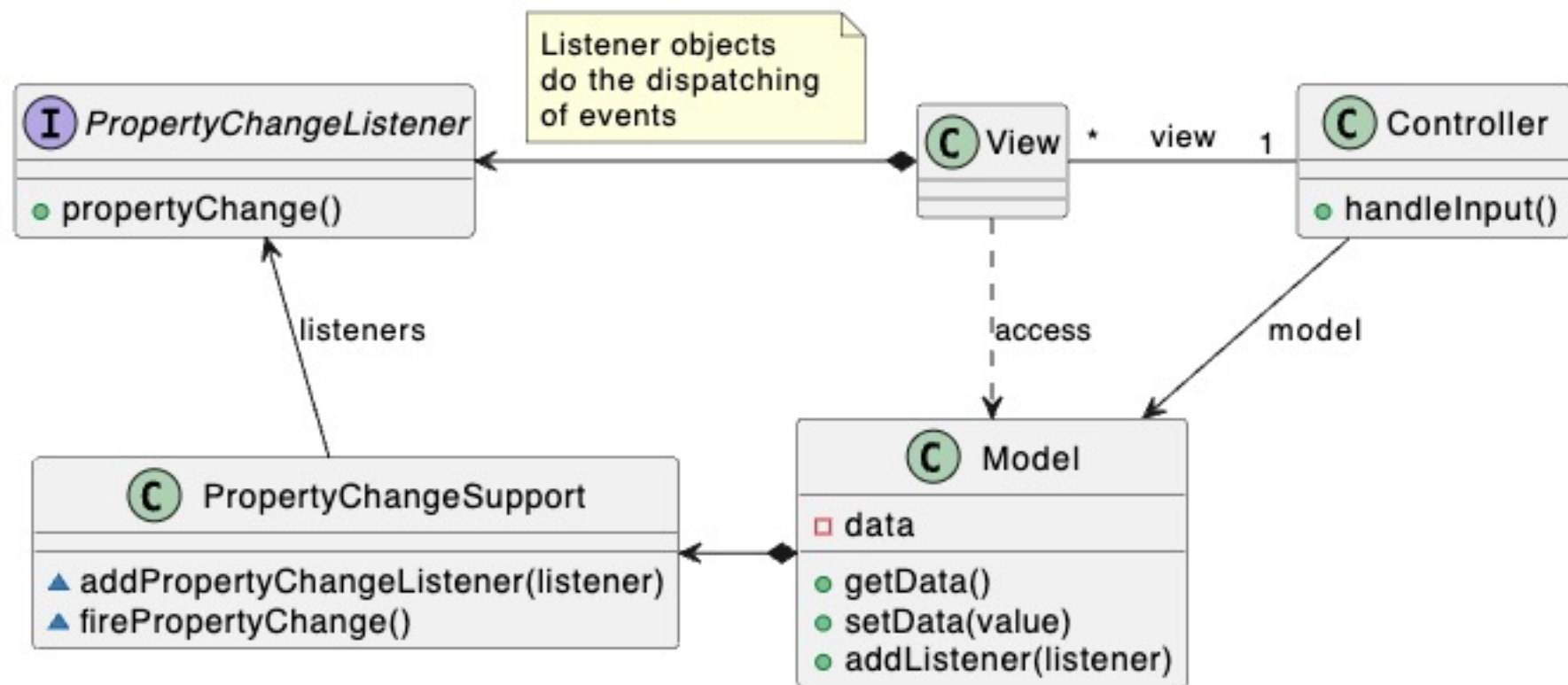
MVP



MVC w/Property Support



MVC w/Property Support



Execution flow

- There is no predefined order of execution
 - Operations are performed in response to external events (e.g. mouse click)
 - Event handling is serialized
 - To execute operations in parallel threads must be used
 - Method main in GUIs has the only goal of instantiating the graphical elements

Summary

- Architectural patterns deal with overall system structure
- They provide a unique metaphor for the system (e.g. pipe and filters)
- They address specific domains (e.g. distribution or interaction) and system evolvability

References

- M.Fowler, K. Scott, UML Distilled, 3rd ed. Addison-Wesley, 2003.
- E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley, 1995.
- E.Freeman, E.Freeman, K.Sierra, B.Bates. Head First Design Patterns, O'Reilly, 2004